



Fake news detection with Amazon Web Service

Realise par :

- Es-saouiqui Amine.
- Boukaidi Tarik .
- Menouar Chaimae .

Encadre par :

- Dr. Routaib Hayat.

Table de contenu :

I.	Introduction :	6
II.	Définitions et Terminologies :	6
1)	Conceptes principaux :	6
	• Fake News Detection :	6
	• NLP (Natural Language Processing)	6
	• Machine Learning (Apprentissage automatique)	7
	• CI/CD (Intégration Continue / Déploiement Continu)	7
2)	Outils utilisés :	7
	GitHub	7
	Scikit-learn	7
	Matplotlib / Seaborn :	7
	NLTK :	7
	Docker :	8
	SageMaker SDK	8
	React :	8
	Vite :	8
3)	Services AWS utilisés :	8
	• Amazon S3 (Simple Storage Service) :	8
	• Amazon SageMaker :	9
	• Amazon ECR (Elastic Container Registry)	9
	• Amazon API Gateway :	9
	• Amazon IAM (Identity and Access Management)	9
4)	Autres Services :	10
	• Github Action :	10
5)	Users, Roles et Politiques :	10
	• User (Utilisateur) :	10
	• Role (Rôle) :	10

•	Policy (Politique / Règle):	10
III.	Architecture du projet :.....	11
1)	– Architecture Generale:.....	11
2)	Architecture Deploiment :	13
IV.	Dataset & Prétraitement:.....	15
1)	Description du dataset :	15
2)	Les etapes de pretrainement :	16
3)	Stockage dans S3 :	16
V.	Modélisation:	20
1)	Choix de modele :	20
2)	Deploiment du modele :	22
•	Cycle de vie du deploiment d'un modele dans Amazon SageMaker :.....	22
•	Deploiment d'un conteneur personnalise :	23
VI.	Configuration du passerelle :	29
1)	Creation d'un API Gateway :	30
VII.	Deploiment de la partie frontale :	35
VIII.	Continous Integration & Continous Deployment :.....	39
1)	Deploiment du modele :	39
•	Workflow -1 :	41
•	Workflow -2 :	44
2)	Deploiment d'application Web :	46
•	Workflow :	47
IX.	Gestion des accès et communication sécurisée entre les services :	49
1)	Creation d'un utilisateur :	50
2)	Creations des roles :	52
3)	Creation des poltiques :	53
•	Assumer le role d'action au SageMaker Role :	53
•	Definir les permissions du role :	55
X.	Conclusion :	57

Table de figures :

Figure 1 : Architecture Generale de la mise en place d'une solution de detection des fake news sur AWS	11
Figure 2 : Architecture Detaille du flux de deployment.	13
Figure 3 : Fake VS Real news	15
Figure 4 : La vue console sur AWS	17
Figure 5 : Barre de recherche de services AWS	17
Figure 6 : Section de gestion des buckets sous S3	18
Figure 7: Configuration Generale de creation des buckets sous S3	18
Figure 8: Appercu sur le contenu ou bien les depots d'un bucket S3	19
Figure 9 : Champ specifier pour nommer le dossier	19
Figure 10 : les fichiers stockes dans le dossier datasets	20
Figure 11 : Les scores atteints par les classifieurs avec les differents metriques	21
Figure 12: Cycle de vie d'un conteneur Sagemaker	22
Figure 13 : Structure hierarchique de dependance d'image	26
Figure 14 : Diagramme de sequence d'une requette HTTP vers SageMaker	30
Figure 15: Section de gestion des passerelles	30
Figure 16: API fournies par API Gateway	31
Figure 17: Formulaire de creation d'un API	32
Figure 18 : Formulaire de creation des ressources	33
Figure 19 : Formulaire de creation d'une methode HTTP	34
Figure 20 : Exemple d'un ressource utilise la methode POST	35
Figure 22 : Page d'accueil	37
Figure 21 : Page d'analyse	37
Figure 23 : reponse d'un real new	38
Figure 24 : Reponse d'un fake new	38
Figure 25 : Le flux automatise pour le deployment d'un modele SageMaker	40
Figure 26 : Section de secrets et variable dans github	40
Figure 27 : Les secrets utilise par le repertoire	41
Figure 28 : L'execution de workflow-1 sous github actions	43
Figure 29 : L'image cree	44
Figure 30 : Registre des images	44
Figure 31 : Model Deploye	46
Figure 32 : l'execution de workflow-2 sous github action	46
Figure 33 : Point de terminaison en Service	46
Figure 34 :Le flux automatise pour le deployment d'une application React dans S3	47
Figure 35 : l'execution de workflow-3 sous github action	48
Figure 36 : Le contenu du frontend	48
Figure 37 : Diagram de gestion d'access AWS du projet	49
Figure 38 : Formulaire de creation des utilisateurs	51
Figure 39 : Formulaire de creation d'un role	53

Table de scriptes :

code 1 : Code d'initiation d'inference	25
code 2 : Configuration d'une image Docker	26
code 3 : Contenu de fichier serve	27
code 4 : Deploiement d'un model serverless dans SageMaker	28
code 5 : Configuration de creation d'image	43
code 6 : Configuration du deployment dans SageMaker	46
code 7 : Configuration du deployment d'pplication React	48
code 8 : politics d'assumption d'un role via STS	54
code 9 : Assumption du role a github workflow action	55
code 10 : Permissions donnees au FakeNewsDetectionRole	56
code 11 : politique d'invocation de sagemaker enpoint	57

I. Introduction :

Les fake news représentent aujourd'hui un problème majeur, car elles se diffusent très rapidement sur les réseaux sociaux et peuvent influencer négativement l'opinion publique. Comme il est impossible de tout vérifier manuellement, l'intelligence artificielle et les techniques de traitement du langage naturel permettent d'automatiser la détection de ces fausses informations.

Amazon Web Services (AWS) propose plusieurs outils utiles pour construire une solution de détection de fake news. Par exemple, Amazon S3 permet de stocker les données, tandis qu'Amazon SageMaker sert à entraîner un modèle de machine learning. Une fois le modèle créé, il peut être déployé sous forme d'API afin d'analyser automatiquement de nouveaux textes. AWS offre également divers services pour transformer ce modèle en une solution web performante et à coût réduit.

Ce rapport explique comment mettre en place une telle solution avec AWS : quelles étapes suivre, quels services utiliser, comment évaluer les performances du modèle, et comment automatiser le flux de déploiement dans le cloud.

II. Définitions et Terminologies :

1) Concepts principaux :

- Fake News Detection :

Processus automatique ou manuel visant à identifier les informations fausses ou trompeuses dans les articles, les publications ou les médias en ligne.

- NLP (Natural Language Processing)

Branche de l'intelligence artificielle qui permet aux ordinateurs de comprendre, analyser et générer du langage humain. Dans ce projet, il est utilisé pour analyser le texte des articles et prédire s'ils sont vrais ou faux.

- **Machine Learning (Apprentissage automatique)**

Technique qui permet à un ordinateur d'apprendre à partir de données pour faire des prédictions ou classifications sans être explicitement programmé pour chaque tâche. Ici, le modèle apprend à distinguer fake news et informations vraies.

- **CI/CD (Intégration Continue / Déploiement Continu)**

C'est une pratique qui permet d'automatiser la mise à jour du code ou des modèles, en testant et en déployant rapidement chaque modification sans intervention manuelle.

2) Outils utilisés :

GitHub

Une plateforme pour héberger et partager du code source. Elle permet de gérer les versions du projet, collaborer avec d'autres développeurs et stocker les notebooks ou scripts utilisés pour entraîner et tester le modèle. Elle intervient aussi dans l'automatisation de test et de déploiement.



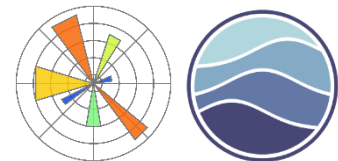
Scikit-learn

Une bibliothèque de machine learning pour Python. Permet de prétraiter les données, transformer du texte en vecteurs, et appliquer des modèles simples (*SVM, Regression Logistic, etc.*).



Matplotlib / Seaborn :

Une bibliothèques pour créer des graphiques et visualiser les résultats, comme les matrices de confusion ou l'évolution des performances du modèle.

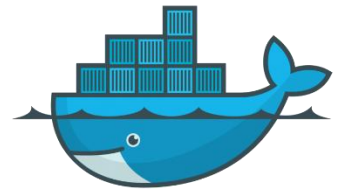


NLTK :

Une bibliothèque Python utilisée pour le traitement du langage naturel. Elle fournit des outils pour nettoyer, tokeniser, analyser et manipuler du texte, ce qui est utile pour préparer les données avant d'entraîner un modèle de détection de fake news.

Docker :

Un outil qui permet de créer des **conteneurs** contenant tout ce dont un programme a besoin pour s'exécuter (*code, bibliothèques, dépendances*) . Cela garantit que le modèle ou l'application fonctionne de la même manière sur n'importe quel ordinateur ou serveur.



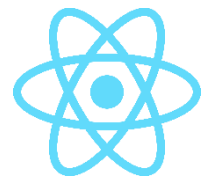
SageMaker SDK

Bibliothèque Python fournie par AWS qui permet d'interagir facilement avec **SageMaker** : créer des jobs d'entraînement, gérer des modèles, déployer des endpoints et faire des prédictions directement depuis Python.



React :

Bibliothèque JavaScript pour créer des interfaces utilisateur interactives et dynamiques, par exemple pour afficher les résultats de prédiction.



Vite :

Outil de développement pour JavaScript et frameworks comme **React**, qui permet de compiler et lancer rapidement une application web. Il accélère le temps de démarrage et de rechargement lors du développement, ce qui rend le travail plus rapide et efficace.



3) Services AWS utilises :

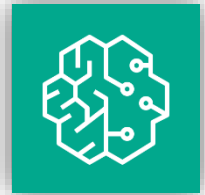
- **Amazon S3 (Simple Storage Service) :**

Service de stockage cloud d'AWS qui permet de sauvegarder et d'organiser des fichiers ou des datasets. Il est utilisé pour stocker les données d'entraînement et les résultats des modèles de manière sécurisée et accessible depuis **SageMaker** même d'héberger la partie frontale de solution web.



- **Amazon SageMaker :**

Service cloud qui permet de créer, entraîner et déployer des modèles de machine learning. Il fournit des notebooks pour développer et tester les modèles, gère les ressources de calcul et facilite la mise en production via des endpoints pour les prédictions.



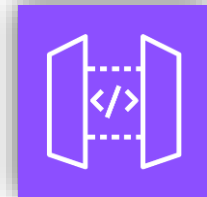
- **Amazon ECR (Elastic Container Registry)**

Service de registre de conteneurs **Docker** sur AWS. Il permet de stocker, gérer et déployer des images Docker qui peuvent ensuite être utilisées pour exécuter des modèles ou des applications dans **SageMaker** ou d'autres services AWS.



- **Amazon API Gateway :**

Service qui permet de créer et gérer des API (*interfaces de programmation*) sécurisées pour vos applications. Dans ton projet, il peut servir à exposer le modèle **SageMaker** comme une API que les utilisateurs ou applications peuvent appeler pour obtenir des prédictions.



- **Amazon IAM (Identity and Access Management)**

Service qui permet de gérer les utilisateurs, les rôles et les permissions dans AWS. Il assure que seules les personnes ou services autorisés peuvent accéder aux ressources comme **S3** , **SageMaker** ou **API Gateway** , garantissant la sécurité de ton projet.



4) Autres Services :

- Github Action :



Service intégré à **GitHub** qui permet d'automatiser des tâches comme tester, construire et déployer du code ou des modèles. Il suit les principes du **CI/CD** , en lançant automatiquement des workflows à chaque modification du dépôt.

5) Users, Roles et Policies :

- User (Utilisateur) :

Une entité **IAM** qui représente une personne ou un service ayant besoin d'accéder aux ressources AWS. Chaque utilisateur peut se voir attribuer des permissions spécifiques pour effectuer certaines actions.

- Role (Rôle) :

Une entité **IAM** que l'on peut attribuer à des utilisateurs, services ou applications pour leur donner des permissions temporaires. Contrairement à un utilisateur, un rôle n'a pas de credentials permanents et est utilisé surtout pour des services ou des scripts automatisés.

- Policy (Politique / Règle):

Document JSON qui définit les permissions (*autoriser ou refuser*) pour un utilisateur, un rôle ou un groupe. Une policy précise quelles actions sont permises sur quelles ressources AWS.

III. Architecture du projet :

1) – Architecture Generale:

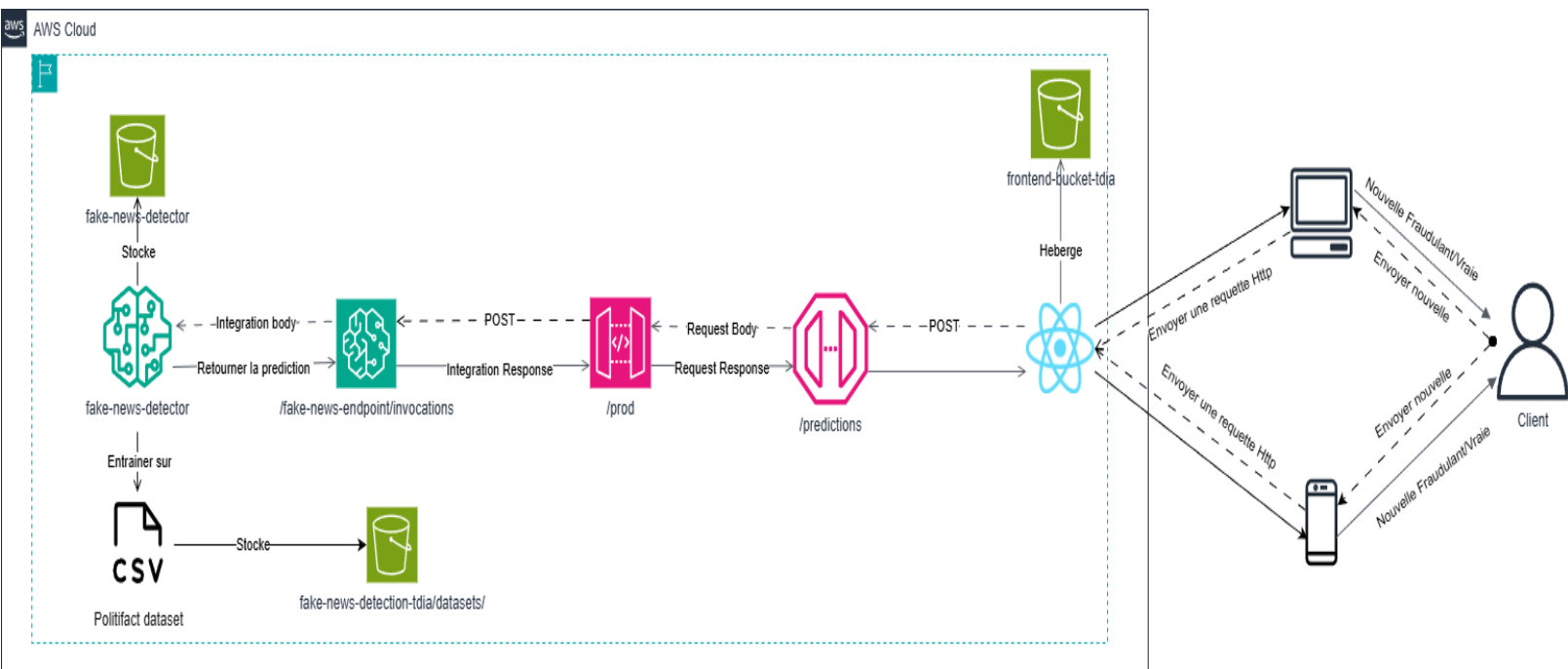


Figure 1 : Architecture Generale de la mise en place d'une solution de detection des fake news sur AWS

Le diagramme ci-dessous présente la vue d'ensemble de l'architecture du projet de détection de fake news.

Il montre la manière dont les différents composants communiquent entre eux, depuis la récupération des données par l'utilisateur jusqu'au retour de la prédiction.

Dans un premier temps, l'utilisateur saisit un texte dans l'application Web développée avec **React**. Cette application envoie la requête à Amazon **API Gateway** en appelant le point de terminaison **/predictions** de l'url fournis avec la methode **POST** du protocole HTTP.

API Gateway joue le rôle d'interface sécurisée entre le front-end et AWS, dans notre architecture, il utilise la représentation **REST** pour communiquer entre le service AWS et le client.

Le point de terminaison prediction a été déployé dans un environnement nommé **/prod** pour indiquer qu'il est le premier à être déployé en production.



La nomenclature des *stages* ou des environnements est arbitraire ; l'important est qu'elle ait une signification claire et partagée au sein de l'équipe.

API Gateway transmet ensuite la requête au point de terminaison déployé sur Amazon **SageMaker**, où le modèle d'analyse de fake news est hébergé. Le modèle exécute l'inférence et renvoie la prédiction : **Vraie** ou **Fraudulent**.

Par ailleurs, le diagramme montre également que les données utilisées pour entraîner le modèle sont stockées dans Amazon **S3** de même que le modèle lui-même.

En raison de problèmes liés à la vérification de compte, nous n'avons pas pu utiliser les services d'hébergement d'applications frontales tels que **CloudFront** ou **Amplify**. L'application a donc été hébergée sur Amazon **S3**, offrant une solution de déploiement à coût réduit et démontrant qu'il ne s'agit pas seulement d'un service de stockage, mais aussi d'un moyen efficace pour héberger des applications statiques comme **React**.

Cette architecture assure à la fois la scalabilité, la sécurité et la rapidité du système.

2) Architecture Deployment :

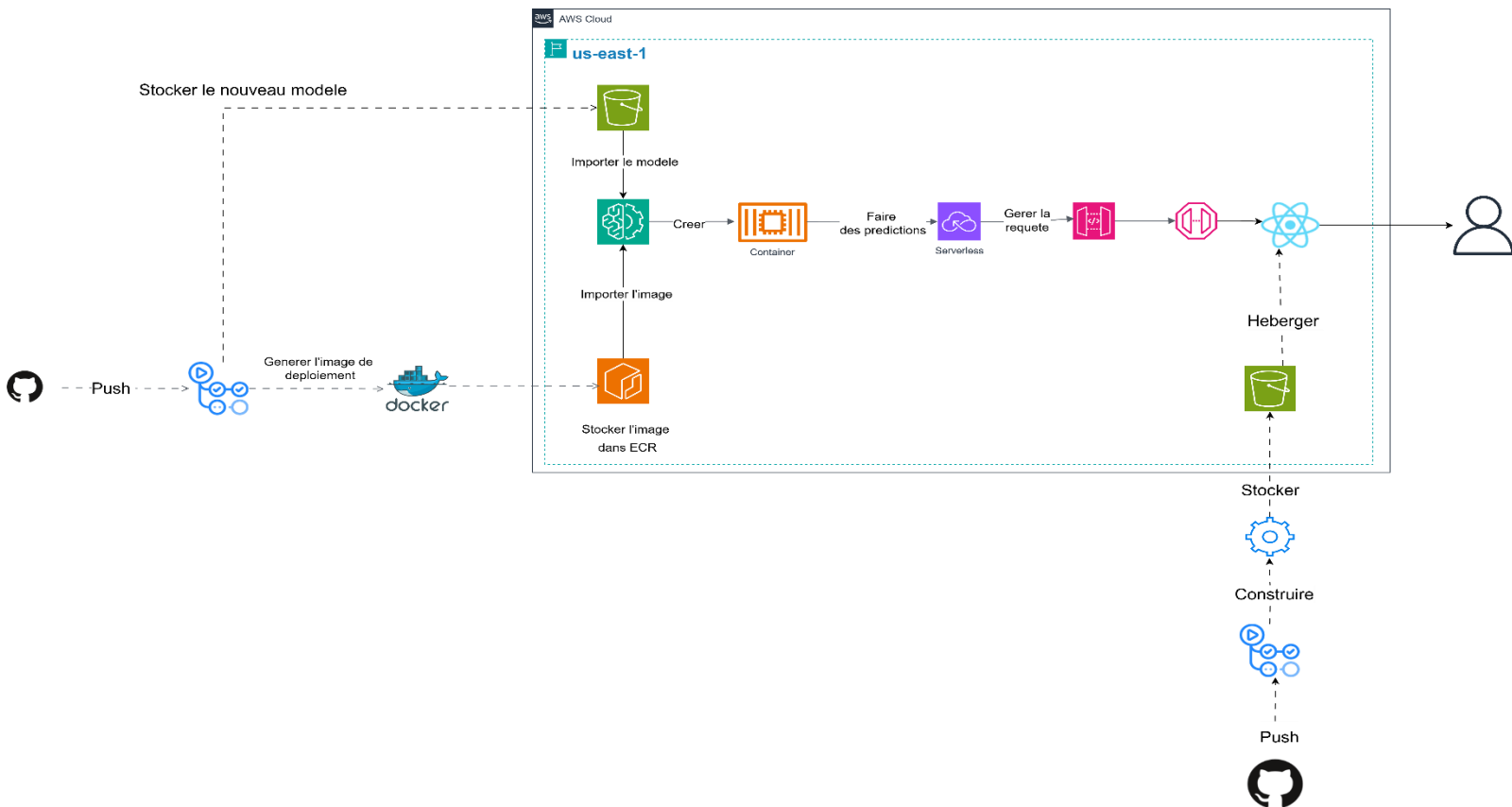


Figure 2 : Architecture Detaillée du flux de deployment.

L'architecture proposée pour le déploiement du modèle de s'appuie sur une chaîne entièrement automatisée, permettant d'assurer la transition entre la phase de développement et la mise en production dans un environnement cloud. Le processus débute par la gestion du code source au sein d'un dépôt **GitHub**, garantissant le versionnement et la collaboration entre les contributeurs.

Chaque mise à jour du dépôt déclenche un pipeline d'intégration et de déploiement continu **CI/CD**, chargé de réaliser deux tâches, la première sert à stocker le nouveau modèle dans **S3**, la deuxième consiste à construire une image **Docker** provisionnant l'environnement idéal pour le déployer ainsi que l'API d'inférence.

Cette image est ensuite transférée vers **ECR**, qui assure un stockage centralisé, sécurisé et optimisé pour les artefacts de conteneurisation.

Le déploiement opérationnel du modèle s'effectue via **SageMaker Serverless Inference**, un service managé permettant d'exécuter des prédictions sans nécessiter la gestion d'une infrastructure sous-jacente. Ce mode de fonctionnement repose sur une allocation dynamique des ressources en fonction de la charge, ce qui garantit une scalabilité automatique et une facturation strictement proportionnelle à l'usage.

L'accès au service d'inférence est assuré par **API Gateway**, qui joue le rôle d'interface sécurisée entre l'application cliente et les services AWS. L'utilisateur interagit avec l'application frontale en transmettant une requête HTTP **POST** vers le point de terminaison de prédiction, et API Gateway se charge de relayer cette requête vers le endpoint **SageMaker** Serverless. Ce dernier exécute l'inférence et renvoie la réponse à l'application, assurant ainsi un flux de communication fiable et performant.

De manière analogue à la partie front-end, chaque mise à jour du dépôt déclenche un pipeline automatisé chargé de reconstruire l'application **React**, de nettoyer les fichiers devenus obsolètes, puis de déployer la version optimisée de l'application sur **S3**.

Dans son ensemble, cette architecture combine les avantages de la conteneurisation, de l'automatisation **CI/CD** et des services managés d'AWS pour offrir une solution robuste, évolutive et économiquement efficiente. Elle facilite la maintenance du système, accélère les cycles de mise à jour du modèle et garantit une disponibilité élevée du service de prédiction, répondant ainsi aux exigences d'un déploiement professionnel de modèles de machine learning.

IV. Dataset & Prétraitement:

1) Description du dataset :

Le jeu de données utilisé pour l'apprentissage était **PolitiFact dataset** contenant des déclarations politiques fact-checkées provenant du site PolitiFact. Chaque entrée inclut le texte de la déclaration ainsi qu'une étiquette indiquant la véracité, soit vrai ou faux. Le dataset est largement utilisé pour entraîner et évaluer des modèles de traitement du langage naturel, notamment pour la détection de fausses informations, la classification automatique de la véracité des déclarations et l'analyse du discours politique. Il constitue une ressource précieuse pour la recherche sur la désinformation et l'intelligence artificielle appliquée à la vérification des faits. Il comprend **17,000** articles entre les nouvelles réelles et frauduleuses.

Il faut noter que l'édition utilise celle de 2023.

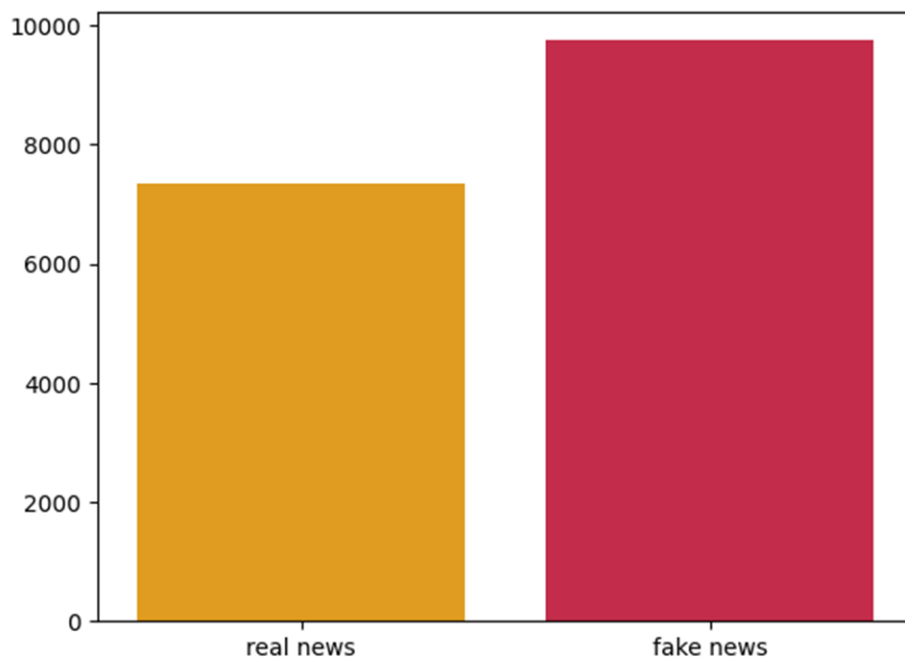


Figure 3 : Fake VS Real news

Comme montre dans la **Figure 3**, on remarque que le nombre des fake news est majoritaire ce qu'il faudra prendre en considération au cours de prétraitement même qu'au cours d'apprentissage.

2) Les étapes de pretraitement :

Avant d'utiliser le **PolitiFact dataset** pour l'entraînement des modèles de classification, un prétraitement complet a été réalisé afin d'assurer la qualité et la cohérence des données textuelles. Tout d'abord, les textes ont été nettoyés à l'aide d'expressions régulières pour supprimer les caractères spéciaux, les chiffres et la ponctuation inutile. Ensuite, la bibliothèque **NLTK** a été utilisée pour tokeniser les textes, retirer les stop words et effectuer la lemmatisation, normalisant ainsi les mots à leur forme de base et réduisant la variabilité du vocabulaire. Enfin, le dataset a été divisé en ensembles d'entraînement et de test, permettant d'évaluer la performance du modèle sur des données non vues. Ce processus garantit que le modèle puisse apprendre efficacement à partir de textes propres et cohérents, tout en minimisant le bruit et les biais liés aux données brutes.

3) Stockage dans S3 :

L'ensemble des données utilisées pour l'entraînement et l'évaluation du modèle de classification a été stocké sur **S3**, qui sert de référentiel centralisé et sécurisé pour le dataset.

Le choix de **S3** permet de bénéficier d'une haute disponibilité, d'une grande durabilité des données ainsi que d'une intégration native avec les autres services AWS, notamment **SageMaker**.

Le version nettoyable du **PolitiFact dataset** a été chargé dans un bucket **S3** afin de faciliter son accès lors des étapes d'entraînement et d'inférence via les différents services cloud.

Pour réaliser cette procédure, il faut comme première étape accéder au service S3 via la console d'AWS.

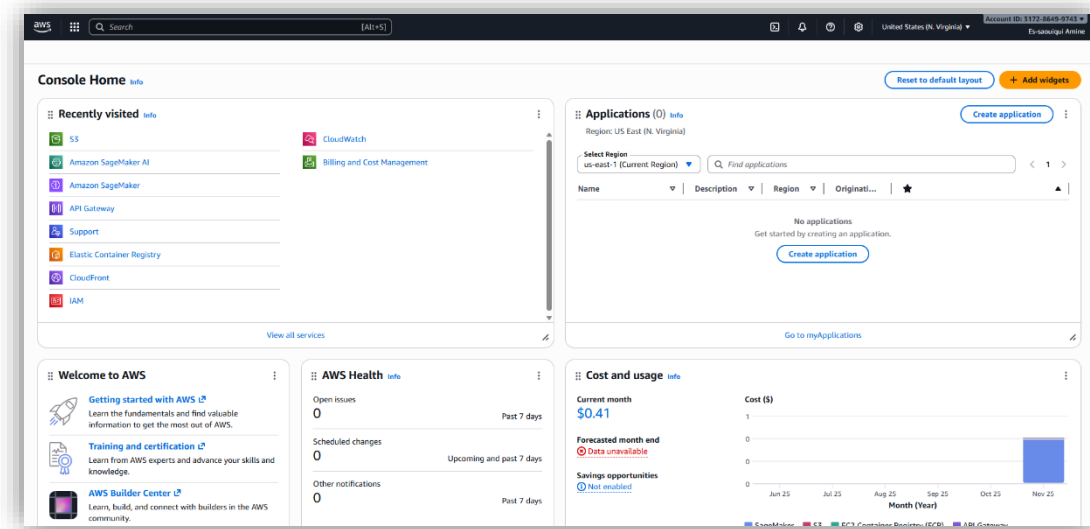


Figure 4 : La vue console sur AWS

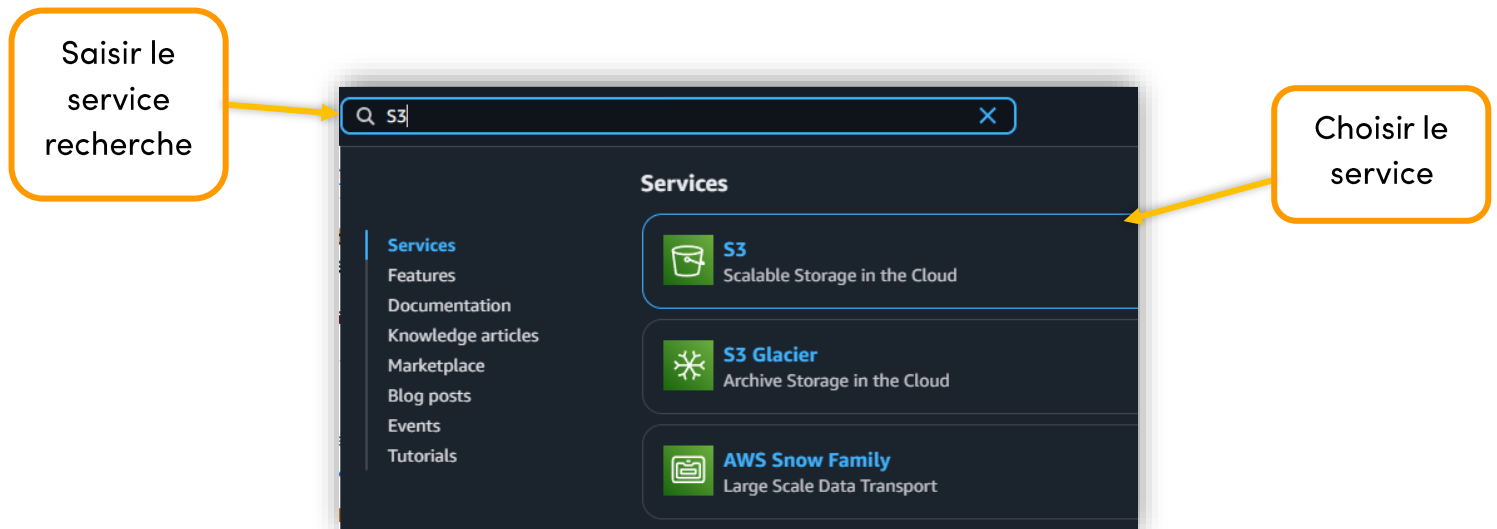


Figure 5 : Barre de recherche de services AWS

Après avoir choisir le service **S3** .La deuxième étape s'agit de créer un bucket en cliquant sur l'option **Create Bucket** .

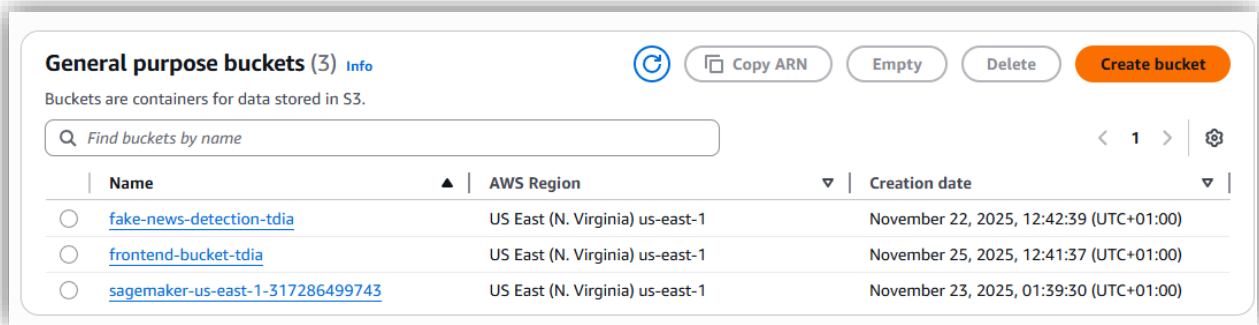


Figure 6 : Section de gestion des buckets sous S3

Une fois cliqué, un formulaire s'affichera dans lequel il devra renseigner un nom unique pour le bucket, pour notre bucket nous avons attribué le nom : **fake-news-detection-tdia**

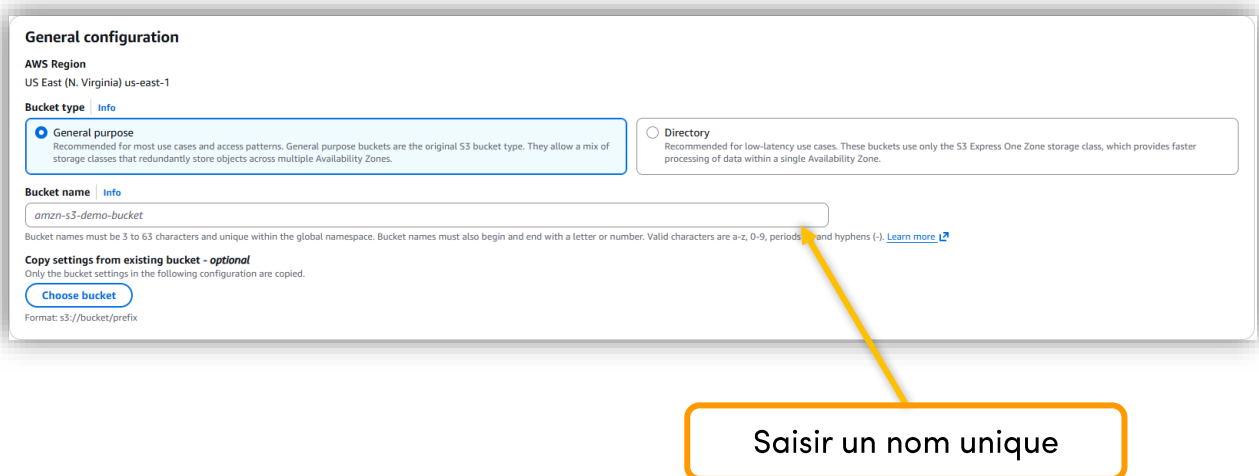


Figure 7: Configuration Générale de création des buckets sous S3

Pour les autres configurations, il est conseillé de les laisser par défaut.

La dernière étape s'agit de cliquer sur **Create Bucket** et voilà le bucket a été bien créé.

Afin de bien structurer le dépôt des jeux de données et du modèle dans l'entrepôt, nous avons adopté une organisation hiérarchique inspirée des systèmes de fichiers. Un dossier dédié au stockage des jeux de données a été créé. Pour ce faire, il suffit de sélectionner l'option **Create Folder** dans le bucket concerné et de lui attribuer un nom.

Le nom de dossier en question a été nommé : **datasets**

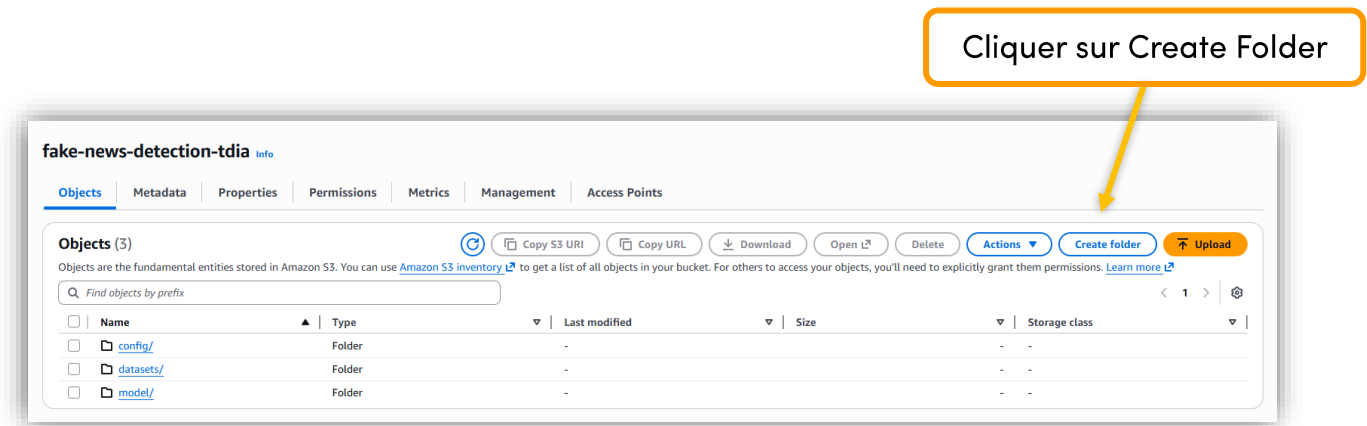


Figure 8: Appercu sur le contenu ou bien les depots d'un bucket S3

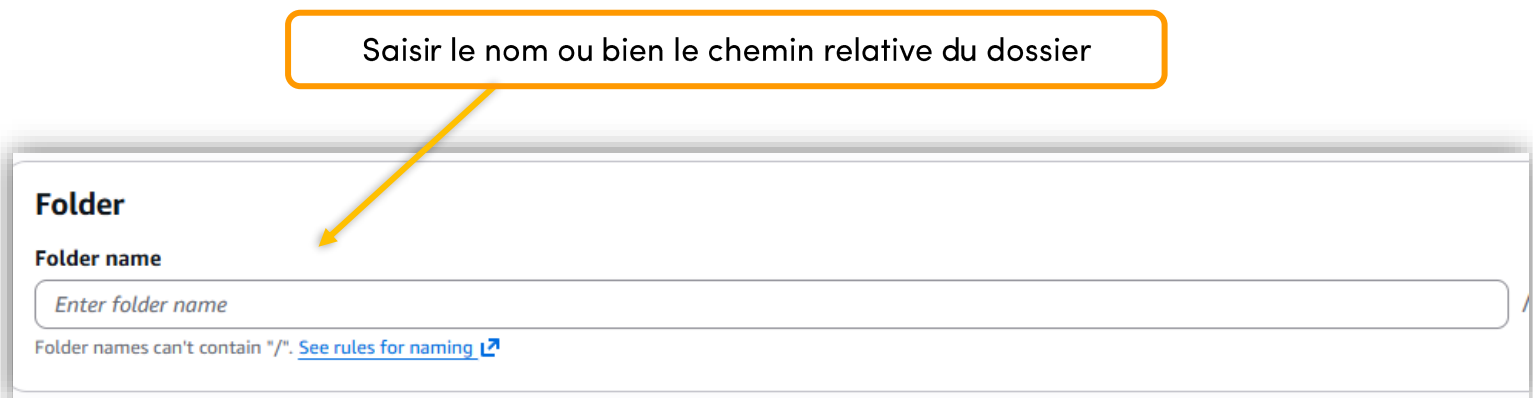


Figure 9 : Champ specifier pour nommer le dossier

La phase finale consiste à placer le dataset préalablement nettoyé dans le répertoire dédié. Pour ce faire, il convient d'accéder au dossier, puis de sélectionner l'option **Upload** afin d'importer le fichier depuis la machine locale vers l'environnement cloud.

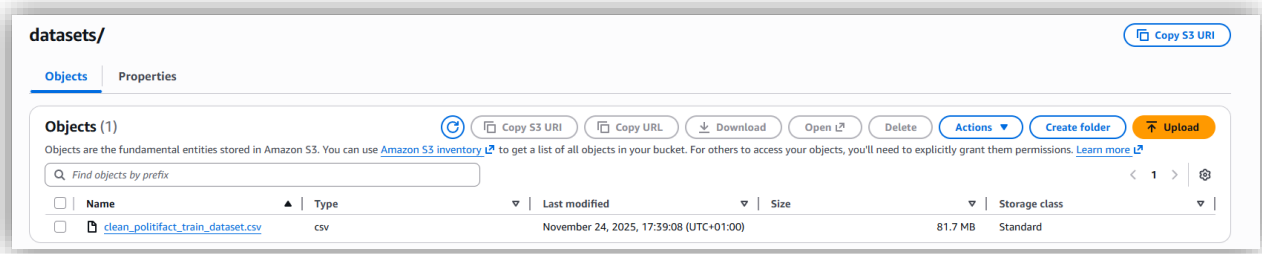


Figure 10 : les fichiers stockés dans le dossier datasets

V. Modélisation:

1) Choix de modèle :

Pour la phase de modélisation, le dataset a été séparé en **données d'entraînement** et **données de test** afin d'évaluer le modèle sur des exemples non vus. Avant l'apprentissage, l'ensemble des textes a été transformé en représentations numériques grâce à la méthode **TF-IDF**, permettant de mettre en valeur les termes les plus pertinents tout en réduisant l'importance des mots fréquents et peu informatifs.

Plusieurs algorithmes de machine learning ont été testés dans le cadre de cette étude afin de comparer leurs performances :

- La régression logistique.
- Naive Bayes.
- K-Nearest Neighbors (KNN).
- La Forêt Aléatoire.

Pour chaque modèle, la recherche des meilleurs hyperparamètres a été réalisée au moyen de **RandomizedSearchCV**, combinée à une validation croisée en 5 folds . Cette approche permet de tester un ensemble varié de combinaisons d'hyperparamètres tout en optimisant le temps de calcul. Étant donné que le dataset présente un déséquilibre important en faveur des fausses informations, la métrique principale choisie pour l'évaluation est le **F1-score**, plus adaptée que la simple précision puisqu'elle prend en compte à la fois le rappel et la précision. Cette stratégie permet d'identifier le modèle le plus robuste face au déséquilibre des classes et le plus performant dans la détection correcte des fausses déclarations.

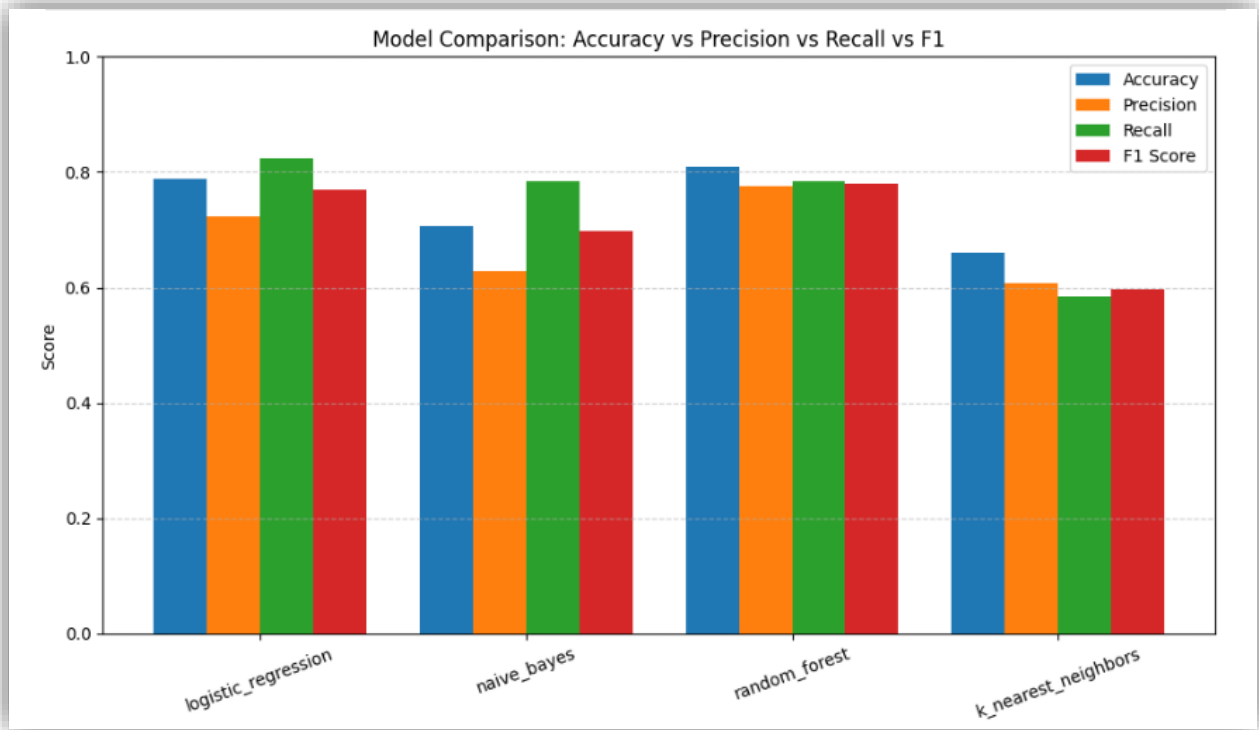


Figure 11 : Les scores atteints par les classifieurs avec les différents metriques

En examinant la

Figure 11, on constate que la forêt aléatoire surpasse légèrement la régression logistique en termes de **F1-score**. Ce modèle offre également un bon compromis entre précision et rappel.

On note aussi que l'ensemble des métriques de la forêt aléatoire se situent autour de **80 %**, ce qui suggère que le modèle n'a pas souffert de sous-apprentissage.



Cependant, bien que ces résultats soient cohérents, ils ne suffisent pas à garantir une véritable capacité de généralisation : seule une évaluation sur un jeu de données totalement indépendant de celui utilisé pour l'entraînement permettrait de confirmer l'absence de sur-apprentissage.

Malgré cette limite, la forêt aléatoire peut être considérée comme une solution prometteuse, offrant des performances encourageantes pour la détection de fausses informations.

2) Déploiement du modèle :

• Cycle de vie du déploiement d'un modèle dans Amazon SageMaker :

Amazon **SageMaker** facilite le déploiement des modèles de manière simple et flexible. Grâce au SageMaker SDK, il suffit de fournir un script d'inférence qui décrit les étapes nécessaires pour charger le modèle, traiter les données d'entrée et produire des sorties adaptées.

En complément, le SageMaker SDK demande également un script de déploiement, qui précise l'emplacement du modèle à charger, le nom du point de terminaison ainsi que le type d'instance de calcul sur lequel le modèle doit être exécuté, sans oublier de deleguer le rôle ayant les privilèges nécessaires pour accéder aux ressources nécessaires pour le déploiement.

Pour mieux comprendre qu'est ce que ce passe en realite,voici la figure ci-dessus:

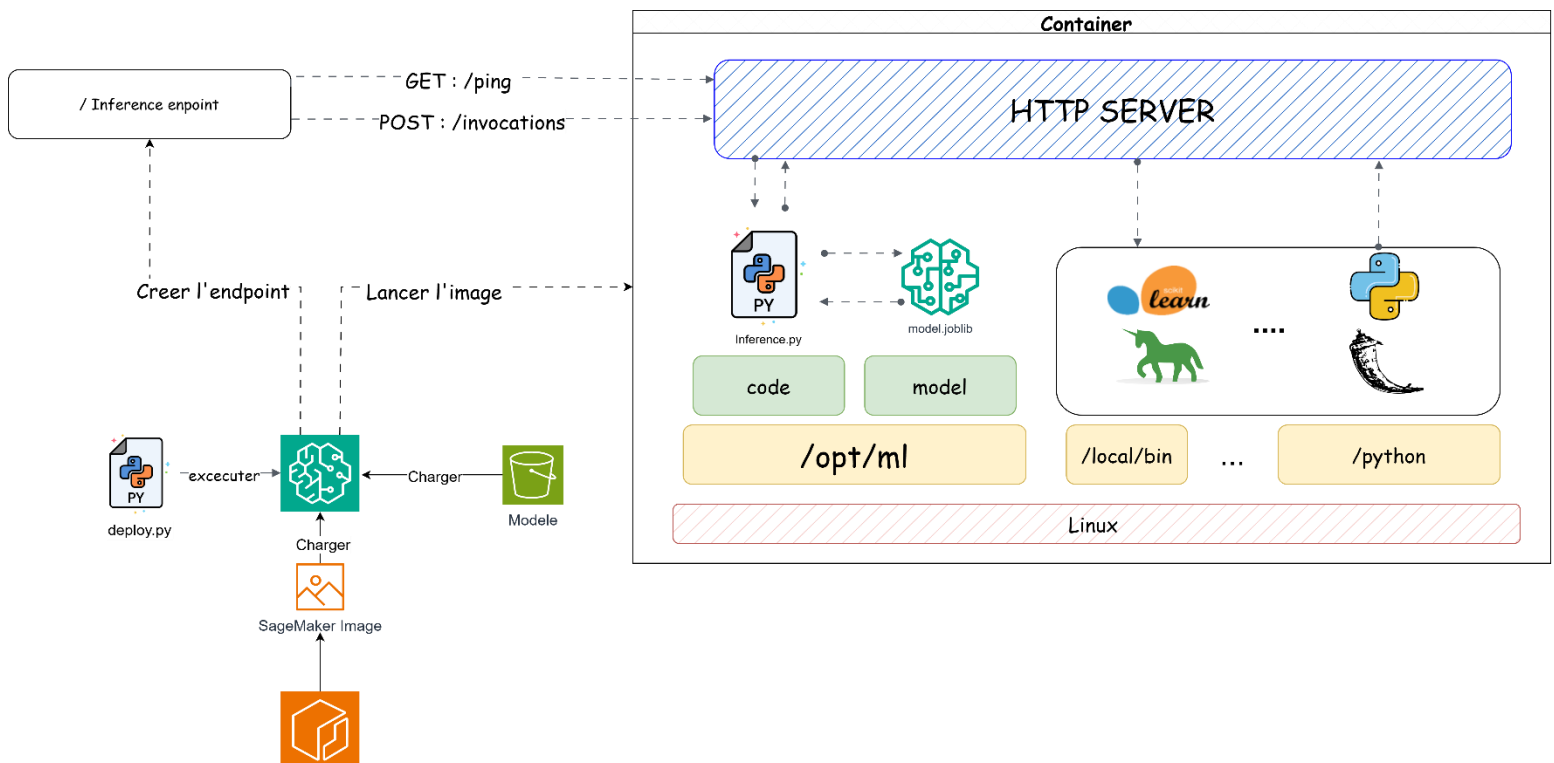


Figure 12: Cycle de vie d'un conteneur Sagemaker

En se basant sur l'architecture présentée, l'exécution du fichier *deploy.py* déclenche la création d'un point de terminaison **SageMaker**. Ce processus implique le chargement de l'image **ECR** dédiée au déploiement du modèle, ainsi que l'importation du modèle entraîné depuis **S3** et du script *inference.py*.

Une fois ces éléments réunis, **SageMaker** lance le conteneur en provisionnant automatiquement les ressources nécessaires à son exécution.

Le modèle est placé dans le répertoire standard **/opt/ml/model**, tandis que le script d'inférence est stocké dans **/opt/ml/code**. Le conteneur repose sur un environnement Linux préconfiguré, intégrant l'interpréteur **Python**, la bibliothèque **scikit-learn**, ainsi que les dépendances requises pour l'inférence.

Afin d'assurer la communication avec le point de terminaison, le conteneur doit embarquer un serveur **HTTP** précisément **Gunicorn** conforme à l'interface **REST**. Celui-ci lance une application **flask** expose deux endpoints essentiels :

- **GET /ping** : utilisé pour vérifier l'état de santé du modèle (health check).
- **POST /invocations** : utilisé pour soumettre des données et obtenir une prédiction.

• **Deploiement d'un conteneur personnalisé :**

Bien que **SageMaker** propose des images prédéfinies, optimisées pour le déploiement de modèles d'apprentissage automatique, il peut arriver que l'on ait besoin d'un **environnement particulier** pour exécuter un modèle. Cela peut concerner, par exemple :

- Les versions spécifiques des dépendances utilisées.
- Le type de serveur **HTTP**.
- Les bibliothèques d'apprentissage automatique qui ne sont pas disponibles dans **SageMaker**.
- Un framework inconnues.
- etc.

Heureusement, AWS permet de créer des **conteneurs personnalisés** répondant à ces besoins spécifiques et de les déployer facilement tout en respectant les normes de déploiement.

Dans notre cas, le modèle a été entraîné avec **Python 3.13**. De plus, les dépendances utilisées par le modèle (*numpy, pandas, scikit-learn, etc.*) sont incompatibles avec les environnements standard fournis par **SageMaker**.

Une deuxième contrainte concerne le prétraitement des entrées . Avant de les passer au modèle, il est nécessaire de nettoyer les données avec **NLTK** . Bien que certaines options puissent être configurées dans les versions fournies par **SageMaker**, nous prévoyons d'utiliser la version **serverless**. Dans ce contexte, les dépendances doivent être chargées très rapidement pour démarrer le conteneur, c'est ce qu'on appelle le **cold start** (*généralement en moins de 180 secondes*) . Il est donc plus efficace de les installer directement dans le conteneur, afin de garantir un démarrage rapide et un traitement optimal.

Avant de créer une image personnalisée pour **SageMaker**, il est nécessaire de respecter certains standards :

- Installer un serveur **HTTP** (par exemple Gunicorn, Uvicorn, etc.) pour gérer les requêtes entrantes.
- Installer un framework web permettant la communication avec le serveur (par exemple Flask ou FastAPI, etc.).
- Implémenter les endpoints obligatoires :
 - **/ping** pour le health check,
 - **/invocations** pour traiter les requêtes d'inférence
- Un fichier exécutable nommé **serve** pour lancer le serveur .

a) – Creation de fichier inference.py :

Comme discuté dans les sections précédentes le fichier **inference.py** joue un rôle central. Il contient la logique nécessaire pour charger le modèle, prétraiter les données d'entrée, effectuer les prédictions et formater les résultats. Dans un conteneur standard **SageMaker**, ce script est appelé automatiquement par le serveur d'inférence intégré, sans que l'utilisateur ait besoin de gérer un serveur web.

Dans notre approche, nous avons initialisé une classe nommée **Cleaner**, dont le rôle est de charger les dépendances **NLTK** et de nettoyer le texte avant de le transmettre au modèle.



```
import os
import joblib
from flask import Flask, request, jsonify
from cleaner import Cleaner

app = Flask(__name__)
cleaner = Cleaner()

# Load the model once at startup
MODEL_NAME = os.getenv('MODEL_NAME')
model_path = os.path.join("/opt/ml/model", MODEL_NAME)
model = joblib.load(model_path)

@app.route("/ping", methods=["GET"])
def ping():
    return "pong", 200

@app.route("/invocations", methods=["POST"])
def invoke():
    if request.content_type != "application/json":
        return jsonify({"error": f"Unsupported content type: {request.content_type}"}) , 400

    input_data = request.get_json()
    text= cleaner.transform(input_data['text'])
    preds = int(model.predict(text)[0])
    confidence_scoe = float(model.predict_proba(text).max())
    return jsonify({"predictions": preds,"confidence_score":confidence_scoe})
```

code 1 : Code d'initiation d'inference

b) – Creation de l'image :

La deuxième étape consiste à créer une image qui sera utilisée pour lancer le conteneur **SageMaker**.

Pour le type d'image, nous avons choisi **Docker**.

La configuration du **Dockerfile** doit inclure les opérations suivantes :

- Copier le fichier d'inférence et le module **Cleaner** dans le conteneur.
- Copier le fichier exécutable du serveur dans **/usr/local/bin/**.
- Installer toutes les dépendances nécessaires pour le modèle.
- Inclure les ressources **NLTK** requises directement dans le conteneur.

La structure de fichier qu'on va copier est comme suit :

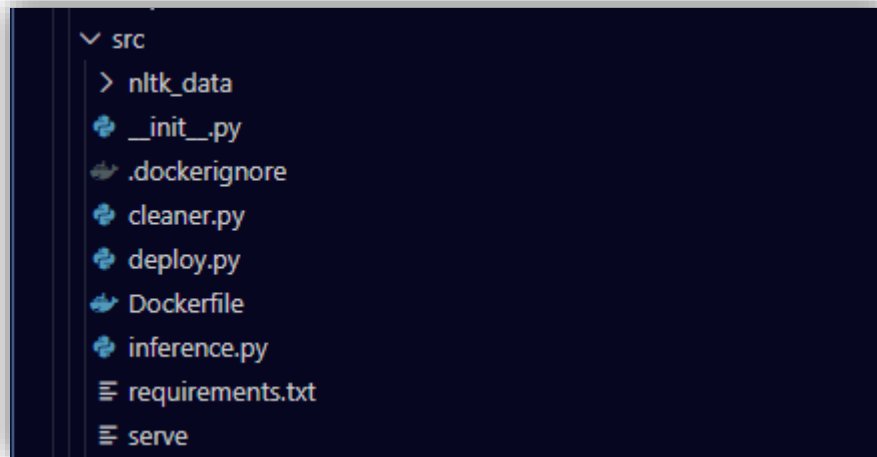


Figure 13 : Structure hierarchique de dependance d'image



Voici la configuration du fichier **Dockerfile** :

```
# using the python3.13 package
FROM python:3.13-slim

# Set working directory
WORKDIR /app

# Copy source code
COPY ML/src/ /app/
COPY ML/src/serve /usr/local/bin/serve

RUN chmod +x /usr/local/bin/serve

# EXPOSE required port
EXPOSE 8080

# Set sagemaker environment variables
ENV PYTHONUNBUFFERED=TRUE
ENV PYTHONDONTWRITEBYTECODE=TRUE
ENV PATH="/app:${PATH}"
# Install dependencies
RUN pip install --upgrade pip
RUN pip install -r requirements.txt
```

code 2 : Configuration d'une image Docker

Le fichier **serve** contient les instructions nécessaires à **SageMaker** pour lancer automatiquement le serveur d'inférence.



```
#!/bin/bash
# Start the server
exec gunicorn --bind 0.0.0.0:8080 inference:app
```

code 3 : Contenu de fichier serve

La dernière étape sert à déposer l'image dans **ECR**, cette étape sera abordée ultérieurement en détails dans les prochaines sections.

c) – Déploiement final du modèle :

Pour déployer le modèle, nous avons défini un fichier **deploy.py** chargé de communiquer avec **SageMaker** via son SDK. Ce script spécifie l'emplacement du modèle, l'URL de l'image Docker utilisée, ainsi que le type de déploiement choisi (classique ou serverless). Il permet également de transmettre les différentes variables d'environnement nécessaires au processus d'inférence, tout en définissant le point de terminaison destiné à recevoir les requêtes.



```
import os
from dotenv import load_dotenv
import sagemaker
from sagemaker.model import Model
from sagemaker.serverless.serverless_inference_config import ServerlessInferenceConfig

load_dotenv()

# Load environment variables
role = os.getenv('ROLE_ARN')
endpoint = os.getenv('SAGEMAKER_ENDPOINT')
ecr_image = os.getenv('IMAGE_URI')
s3_model_path = os.getenv('MODEL_OUTPUT')
model_data = os.path.join(s3_model_path.rstrip('/'), 'pipeline_model.tar.gz')
source_dir = "ML/src"
model_name = "fake-news-detector"

# Create SageMaker session
session = sagemaker.Session()
```

```
# set the configuration of the container
sk_model = Model(
    name=model_name,
    image_uri=ecr_image,
    model_data=model_data,
    role=role,
    entry_point="inference.py",
    source_dir=source_dir,
    env={
        "MODEL_NAME": "pipeline_model.joblib"
    },
    sagemaker_session=session
)

# Configure serverless inference
serverless_config = ServerlessInferenceConfig(
    memory_size_in_mb=2048,
    max_concurrency=5
)

# Deploy the model
predictor = sk_model.deploy(
    endpoint_name=endpoint,
    serverless_inference_config=serverless_config
)

print(f"Model deployed at endpoint: {endpoint}")
```

code 4 : Deploiement d'un model serverless dans SageMaker



La création de l'image, son stockage ainsi que le déploiement du modèle sont entièrement automatisés. Leur implémentation sera détaillée dans les sections suivantes.

VI. Configuration du passerelle :

l'étape suivante consiste à exposer une API publique ou sécurisée permettant à des applications externes d'interagir avec le modèle. Pour cela, nous utilisons **API Gateway** qui permet de créer et gérer des points d'accès **REST**.

API Gateway agit comme une **couche d'accès** située entre l'utilisateur final et le point de terminaison **SageMaker**. Son rôle principal est de recevoir les requêtes, d'appliquer éventuellement des règles de sécurité, puis de transmettre la requête au modèle **SageMaker**.

Grâce à cette architecture, l'application cliente n'appelle jamais directement le point de terminaison **SageMaker**, c'est **API Gateway** qui joue l'intermédiaire, ce qui renforce la sécurité, la flexibilité et le contrôle des accès.

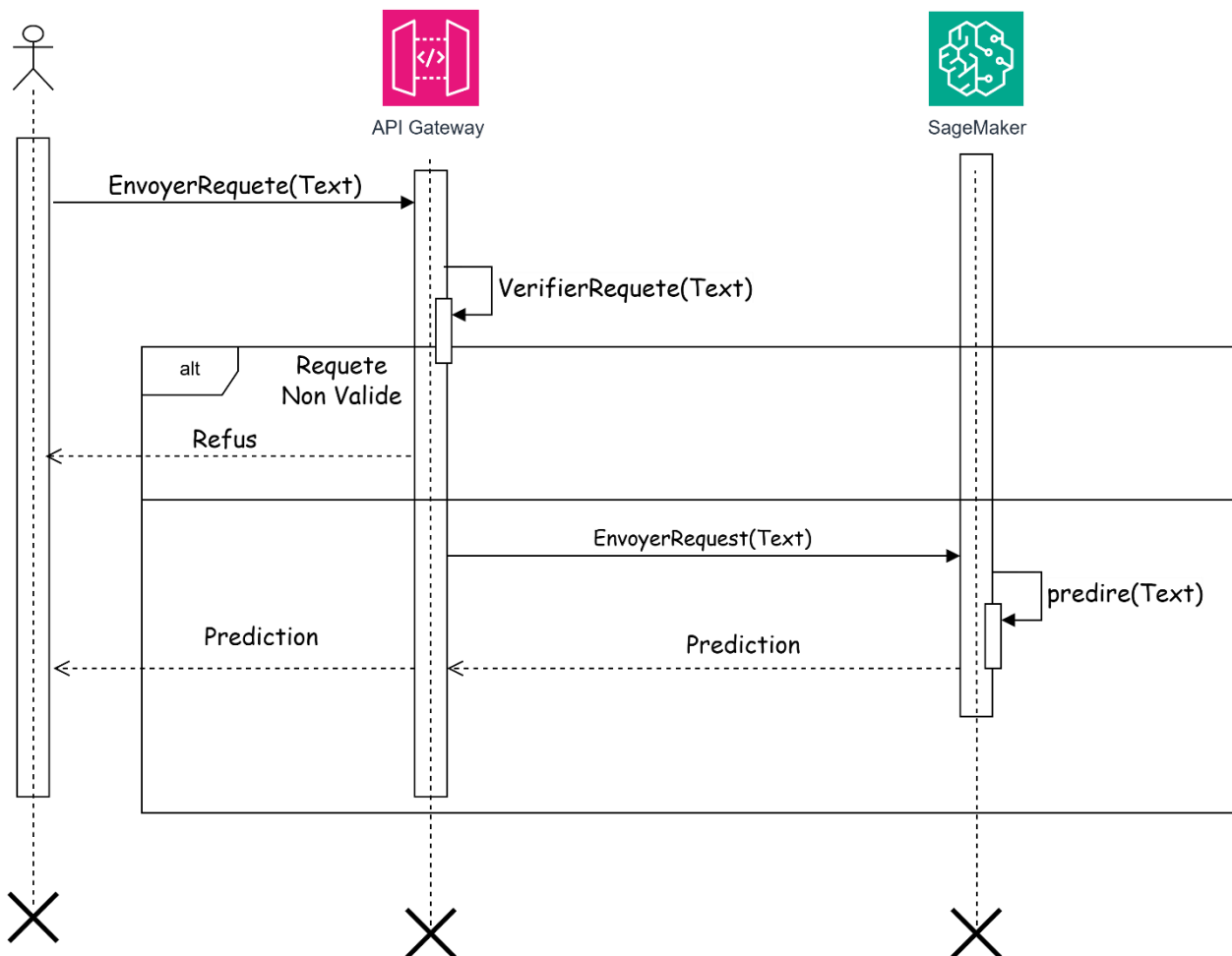
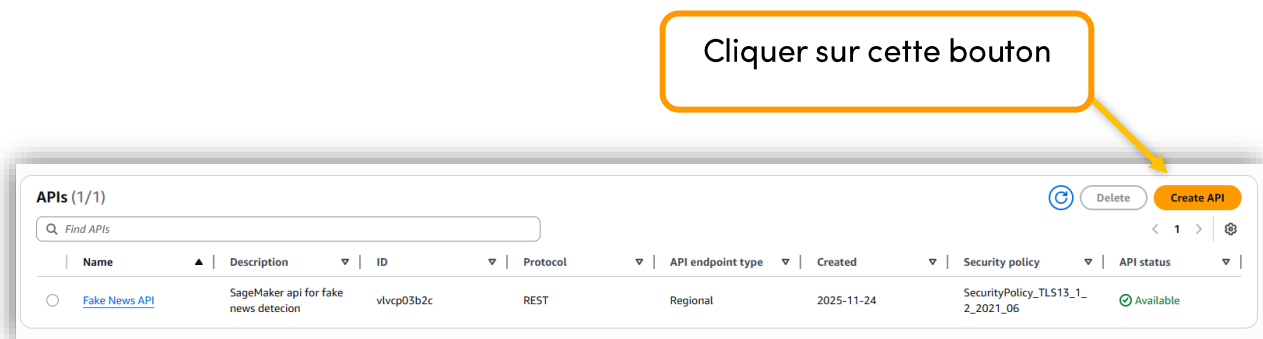


Figure 14 : Diagramme de sequence d'une requette HTTP vers SageMaker

1) Creation d'un API Gateway :

Similairement a ce que nous avons applique pour creer un Bucket S3 [Figure 5], On va chercher API Gateway dans le console .

Une fois que la page de service s'afficher, on clique sur **Create API** pour creer une passerelle.

*Figure 15: Section de gestion des passerelles*

Une fois clique, il faudra choisir le type d'API a utiliser, pour notre approche nous avons choisi **API REST**.

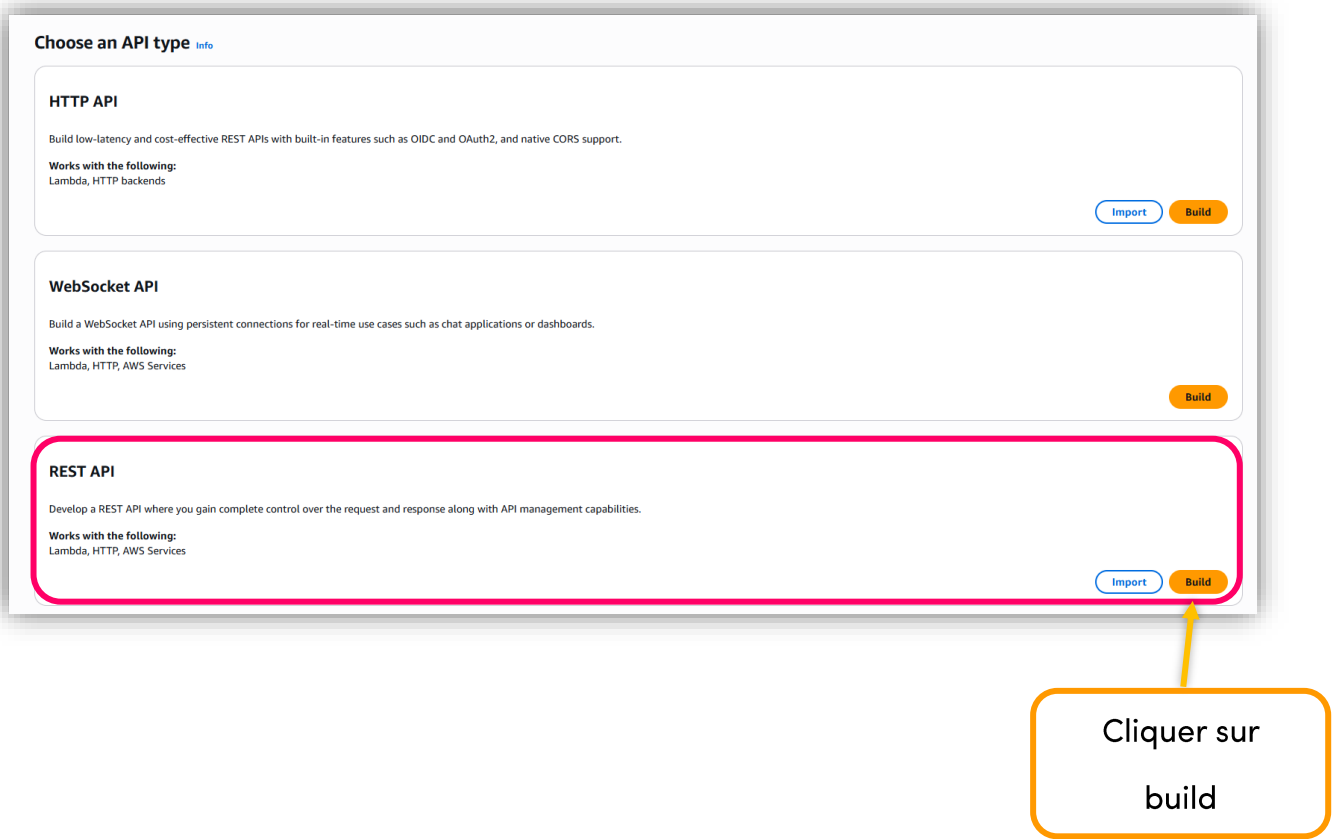


Figure 16: API fournies par API Gateway

L'étape suivante consiste à configurer notre API, les figures ci-dessous montrent le processus de configuration :

The image shows the 'API details' form in the AWS API Gateway console. It includes several sections: 'New API' (selected), 'Clone existing API', 'Import API', and 'Example API'. The 'API name' field contains 'My REST API'. The 'Description - optional' field is empty. The 'API endpoint type' dropdown is set to 'Regional'. The 'Security policy' dropdown is set to 'Choose a security policy'. The 'IP address type' section has 'IPv4' selected. The 'Create API' button is highlighted with a red box. Annotations with orange arrows point to the 'New API' button, the 'API name' field, the 'API endpoint type' dropdown, the 'IPv4' radio button, and the 'Create API' button.

Selectionner New API

Nommer l'API

API details

☒ New API
Create a new REST API.

☐ Clone existing API
Create a copy of an API in this AWS account.

☐ Import API
Import an API from an OpenAPI definition.

☐ Example API
Learn about API Gateway with an example API.

API name
My REST API

Description - optional

API endpoint type
Regional

Security policy - new | info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
Choose a security policy

IP address type | info
Select the type of IP addresses that can invoke the default endpoint for your API.
☒ IPv4
Supports only edge-optimized and Regional API endpoint types.
☐ Dualstack
Supports all API endpoint types.

Cancel Create API

Type d'adress IP

Selectionner le type

Create API

Figure 17: Formulaire de creation d'un API

Après avoir créer la passerelle, on passe à la création de ressource qui présente une partie du chemin de l'API qui sert à organiser et structurer les endpoints.

- Chaque ressource peut contenir une ou plusieurs méthodes (GET, POST, DELETE, etc.).

Pour créer une ressource il suffit de cliquer sur **Create Resource**, puis spécifier le chemin et le nom de la ressource.

Dans notre projet, la ressource a été nommée : **/predictions**

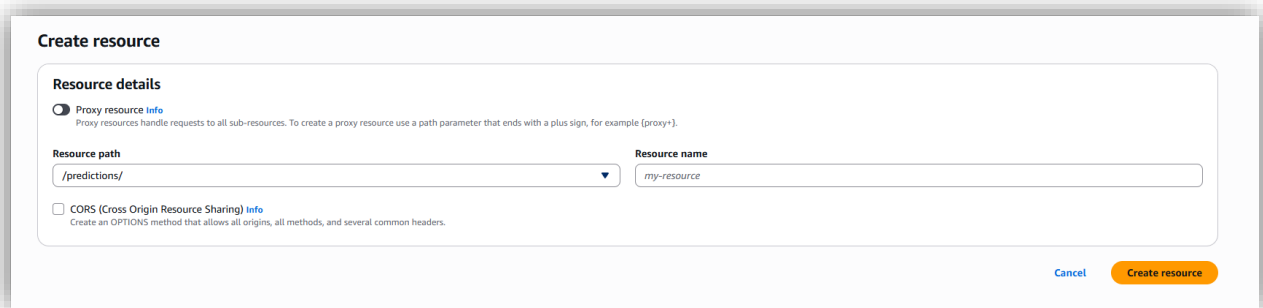


Figure 18 : Formulaire de creation des ressources

Comme déjà mentionné [Page 4] le client envoie une requête en utilisant la méthode **POST**. Il est donc nécessaire d'en créer une. Pour ce faire, il suffit de sélectionner la ressource concernée puis de cliquer sur **Create Method**. ensuite il faut suivre les etapes suivantes :

- Saisir la methode utilise par le client (POST).
- Choisir type d'integration (service).
- Specifier la region.
- Le service avec quel on doit communiquer.
- La methode a utiliser pour envoyer une requete vers le service.
- Le chemin de service.
- Le role d'execution pour executer l'action de prediction de SageMaker par API gateway lui meme.

Concernant les paramètres de la requête (*Method Request Settings*), puisque l'accès à l'API est public, aucune configuration supplémentaire n'est nécessaire. Aucun paramètre n'est à transmettre dans l'URL, ni d'informations additionnelles à inclure dans le header, ce qui justifie l'absence de configuration des deux dernières options. Bien qu'il soit recommandé de schématiser le format des données envoyées, dans notre cas l'entrée correspond simplement à un champ textuel, ce qui rend son passage direct tout à fait approprié.

Create method

Method details

Method type
POST

Integration type

☐ Lambda function
Integrate your API with a Lambda function.

☐ HTTP
Integrate with an existing HTTP endpoint.

☐ Mock
Generate a response based on API Gateway mappings and transformations.

☒ AWS service
Integrate with an AWS Service.

☐ VPC link
Integrate with a resource that isn't accessible over the public internet.

AWS Region
us-east-1

AWS service
SageMaker

AWS subdomain

HTTP method
POST

Action type
☐ Use action name
☒ Use path override

Path override
rest/resource/path

Execution role
arn:aws:iam::myAccount:role/myRole

Credential cache
Do not add caller credentials to cache key

Content handling [Learn more](#)
Passthrough

Integration timeout [Info](#)
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms.
29000

► Method request settings

► URL query string parameters

► HTTP request headers

► Request body

[Cancel](#) [Create method](#)

Figure 19 : Formulaire de creation d'une methode HTTP

Une fois termine il suffit de deployer la ressource en cliquant sur **Deploy API** .

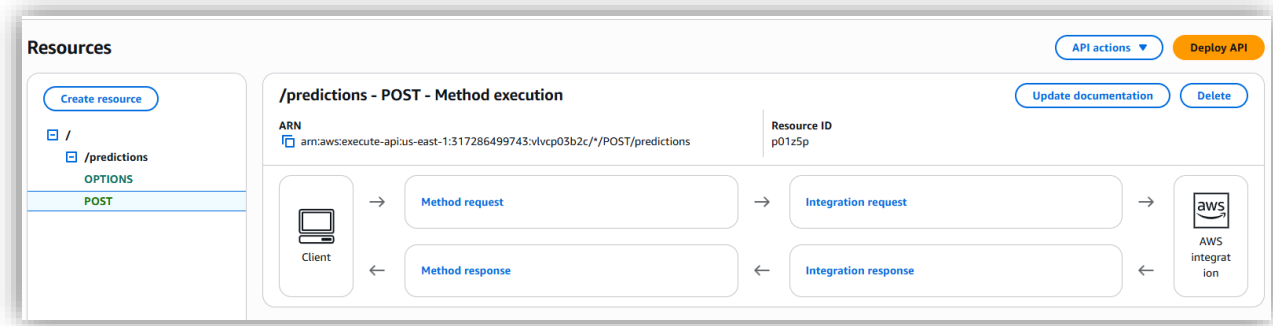
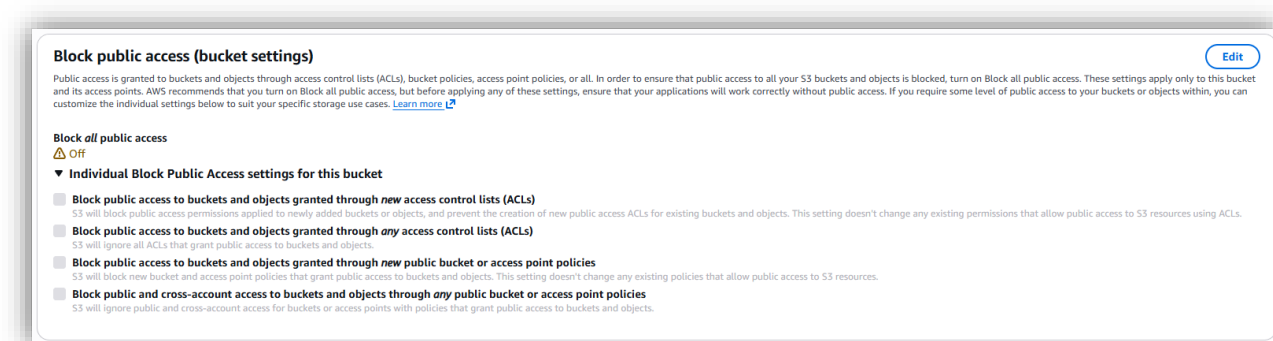


Figure 20 : Exemple d'un ressource utilise la methode POST

VII. Deploiement de la partie frontale :

La partie frontale de l'application se limite à une page statique développée avec React. Celle-ci accepte un texte en entrée et transmet une requête via la méthode **POST** à l'API exposée par API Gateway. En raison de contraintes liées à la vérification de compte, il n'a pas été possible d'utiliser les services d'hébergement web tels que CloudFront ou Amplify. Toutefois, étant donné la légèreté de l'application, nous avons opté pour un hébergement sur un bucket S3, ce qui permet de déployer des solutions web minimales avec un coût réduit.

Pour héberger une page dans Amazon S3, il est nécessaire de créer un bucket, d'autoriser l'accès public, d'activer l'option d'hébergement statique, puis de définir une politique d'accès aux objets permettant leur consultation par tous.



Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

[Edit](#)

We recommend using AWS Amplify Hosting for static website hosting

Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. Learn more about [Amplify Hosting](#) or [View your existing Amplify apps](#)

[Create Amplify app](#)

S3 static website hosting

Enabled

Hosting type

Bucket hosting

Bucket website endpoint

When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)

<http://frontend-bucket-tdia.s3-website-us-east-1.amazonaws.com>

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

[Edit](#)[Delete](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::frontend-bucket-tdia/*"
    }
  ]
}
```

[Copy](#)

La construction de l'application et sa deployment ont aussi automatisés



Figure 22 : Page d'accueil

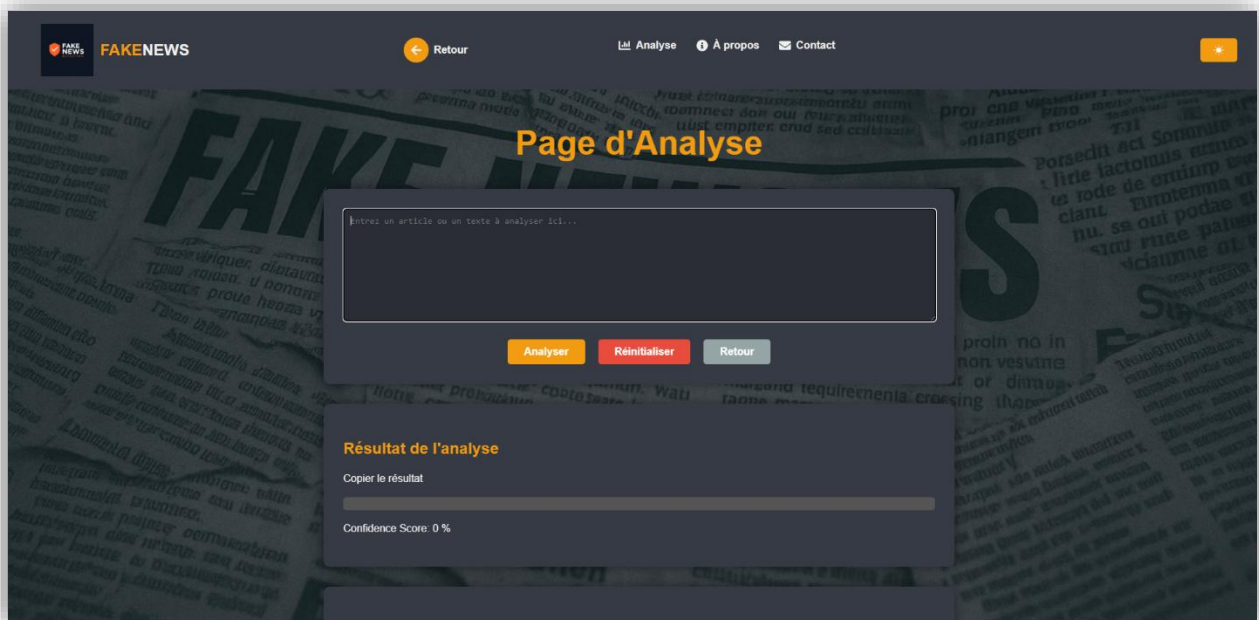


Figure 21 : Page d'analyse

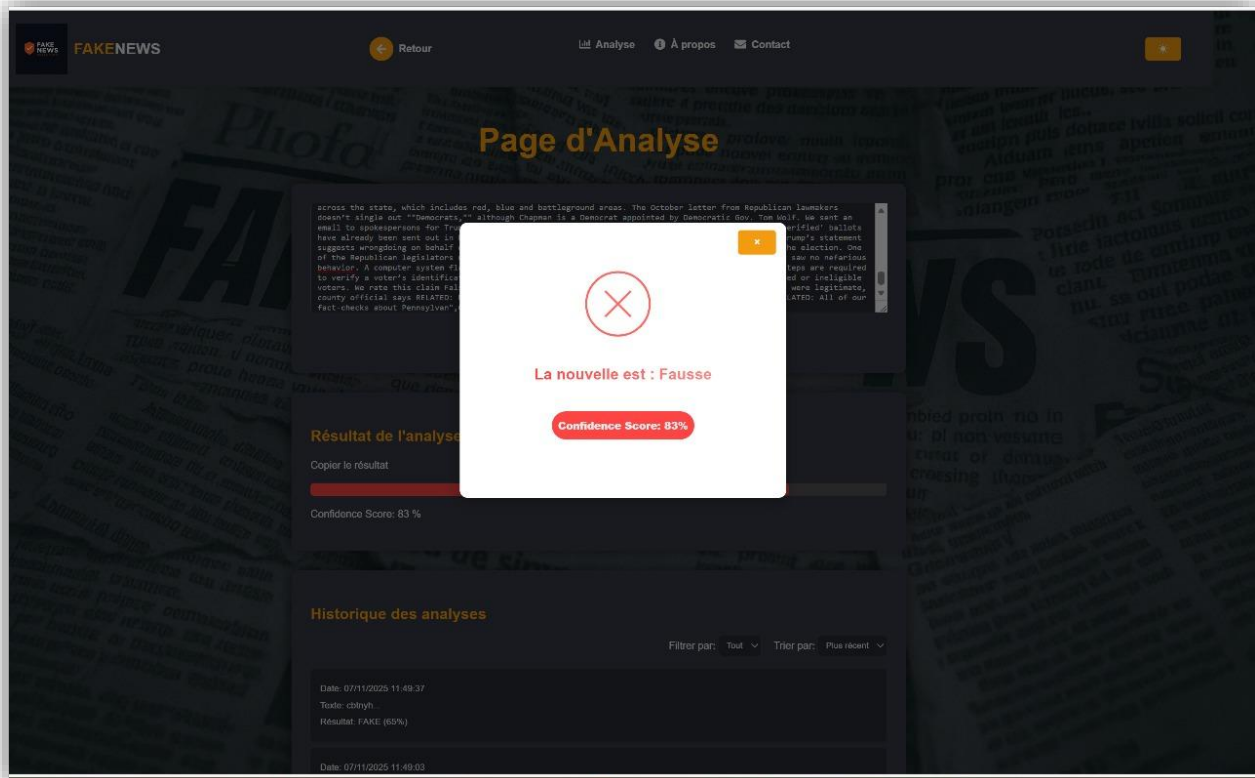


Figure 24 : Reponse d'un fake new

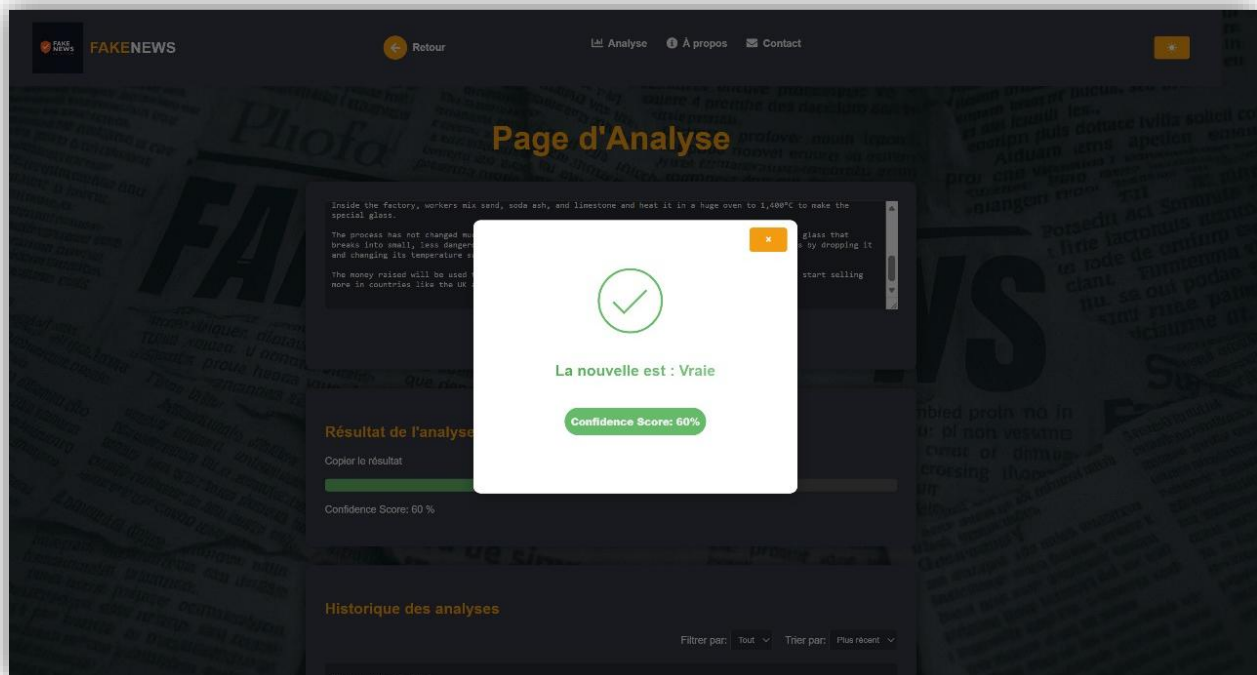


Figure 23 : reponse d'un real new

VIII. Continuous Integration & Continuous Deployment :

Après avoir finalisé le développement du modèle et validé son fonctionnement dans un environnement manuel, nous entamons désormais une étape déterminante : **l'automatisation du déploiement**.

Cette phase vise à industrialiser l'ensemble du processus, aussi bien pour le modèle d'apprentissage automatique que pour l'application **React** qui l'exploite. L'objectif est de rendre les mises à jour plus rapides, plus fiables et totalement reproductibles.

Pour cela, nous mettons en place un pipeline **CI/CD** capable d'automatiser la construction de l'image **Docker**, l'exécution des tests, la publication du conteneur dans **ECR** ainsi que la mise à jour du point de terminaison **SageMaker**. Parallèlement, nous automatisons également le déploiement de l'application **React**, de manière à ce que toute modification du code soit automatiquement buildée puis publiée sur l'environnement d'hébergement. Ainsi, chaque évolution du projet, qu'elle concerne l'interface utilisateur ou le modèle, déclenchera un cycle d'intégration et de déploiement continu, minimisant les erreurs humaines et garantissant une mise en production cohérente, stable et efficace.

1) Déploiement du modèle :

Le premier processus de déploiement concerne exclusivement le modèle, et il est organisé en deux workflows **GitHub actions** complémentaires.

- Le **premier workflow** est chargé de construire l'image **Docker** contenant le script d'inférence et toutes les dépendances nécessaires, puis de la publier dans le registre privé **Amazon ECR**. Une fois l'image stockée, son URL est automatiquement extraite et transmise au second workflow.
- Le **second workflow** reçoit cette URL, installe le **SDK SageMaker** et prépare les différentes variables requises pour le déploiement (*nom du modèle, type d'instance, rôle d'exécution, etc.*). Il exécute ensuite le script **deploy.py**, lequel se charge de créer l'objet *Model* dans **SageMaker** et de déployer cette nouvelle version dans l'environnement cible, notamment en la synchronisant avec **S3** si nécessaire.

Ce mécanisme en deux étapes assure une séparation claire entre la phase de construction de l'image et la phase de déploiement opérationnel du modèle.

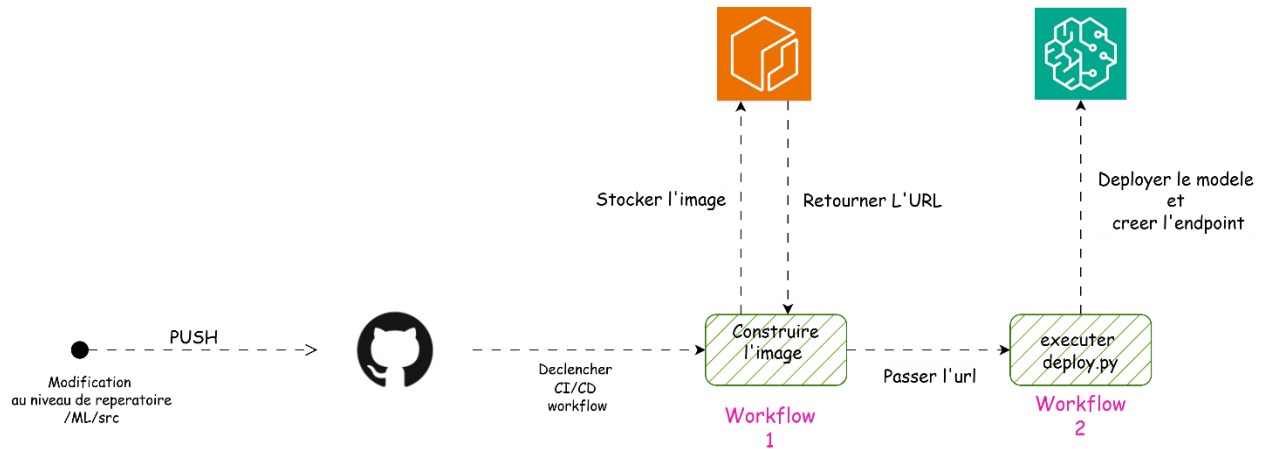


Figure 25 : Le flux automatise pour le deployment d'un modele SageMaker

Pour configurer un github action il faut creer un fichier YAML dans `.github/workflow` decrivant l'execution du processus.

Dans le cadre de l'automatisation du deploiement, il est indispensable de manipuler des informations sensibles telles que les identifiants AWS, les rôles IAM, les noms de buckets ou encore l'URL des ressources. **GitHub Actions** ne doit jamais contenir ces informations en clair dans le code. Pour garantir la sécurité du pipeline, nous utilisons les **GitHub Secrets**, un mécanisme intégré de **GitHub** permettant de stocker et protéger les variables sensibles.

La premiere etape a realiser c'est de definir le secrets, pour ce faire, il faut aller au repertoire github contenant le projet et acceder vers **Settings**, puis aller vers la section **Secrets & variables** de l'onglet **Security** et cliquer sur **Actions**.

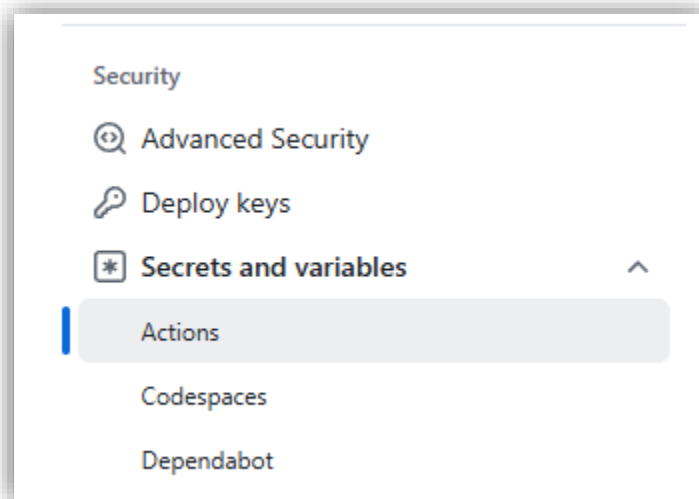
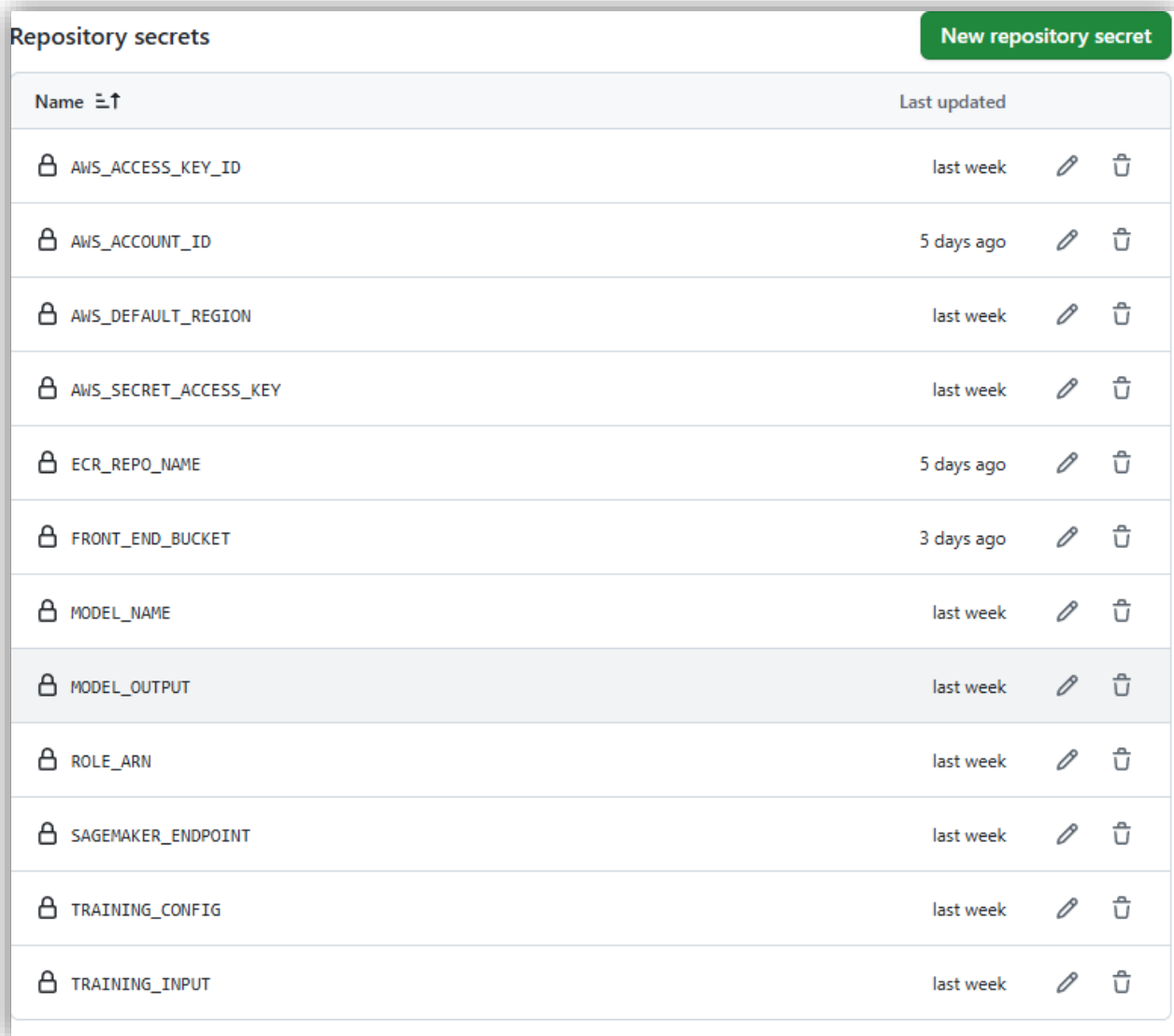


Figure 26 : Section de secrets et variable dans github

Puis definir les informations a transmettre dans des variables :



Repository secrets		New repository secret	
Name 	Last updated		
 AWS_ACCESS_KEY_ID	last week		
 AWS_ACCOUNT_ID	5 days ago		
 AWS_DEFAULT_REGION	last week		
 AWS_SECRET_ACCESS_KEY	last week		
 ECR_REPO_NAME	5 days ago		
 FRONT_END_BUCKET	3 days ago		
 MODEL_NAME	last week		
 MODEL_OUTPUT	last week		
 ROLE_ARN	last week		
 SAGEMAKER_ENDPOINT	last week		
 TRAINING_CONFIG	last week		
 TRAINING_INPUT	last week		

Figure 27 : Les secrets utilise par le repertoire

- Workflow -1 :



```

name: Sage Maker Custom Container
on:
  push:
    branches:
      - master
    paths:
      - "ML/src/**"
jobs:
  build-and-push:
    runs-on: ubuntu-latest

# For OCID authorization
permissions:
  id-token: write
  contents: read

steps:
  - name: Checkout repository
    uses: actions/checkout@v3

  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v3
    with:
      aws-region: ${ secrets.AWS_DEFAULT_REGION }
      role-to-assume: ${ secrets.ROLE_ARN }

  - name: Debug IAM user
    run: aws sts get-caller-identity

  - name: Login to ECR
    id: login-ecr
    uses: aws-actions/amazon-ecr-login@v1

  - name: Build Docker image
    run: |
      IMAGE_URI="${ steps.login-ecr.outputs.registry }/${ secrets.ECR_REPO_NAME
}}:latest"
      echo "Building Docker image: $IMAGE_URI"
      docker build -t $IMAGE_URI -f ML/src/Dockerfile .

      # Set environment variable for this w
      echo "IMAGE_URI=$IMAGE_URI" >> $GITHUB_ENV

  - name: Save the IMAGE_URI to JSON
    run: |
      echo "{\"image_uri\":\"$IMAGE_URI\"}" >> image.json

  - name: Save the file to an artifact
    uses: actions/upload-artifact@v4
    with:
      name: image_info.json
      path: image.json

  - name: Push Docker image
    run: docker push $IMAGE_URI

```

code 5 : Configuration de creation d'image

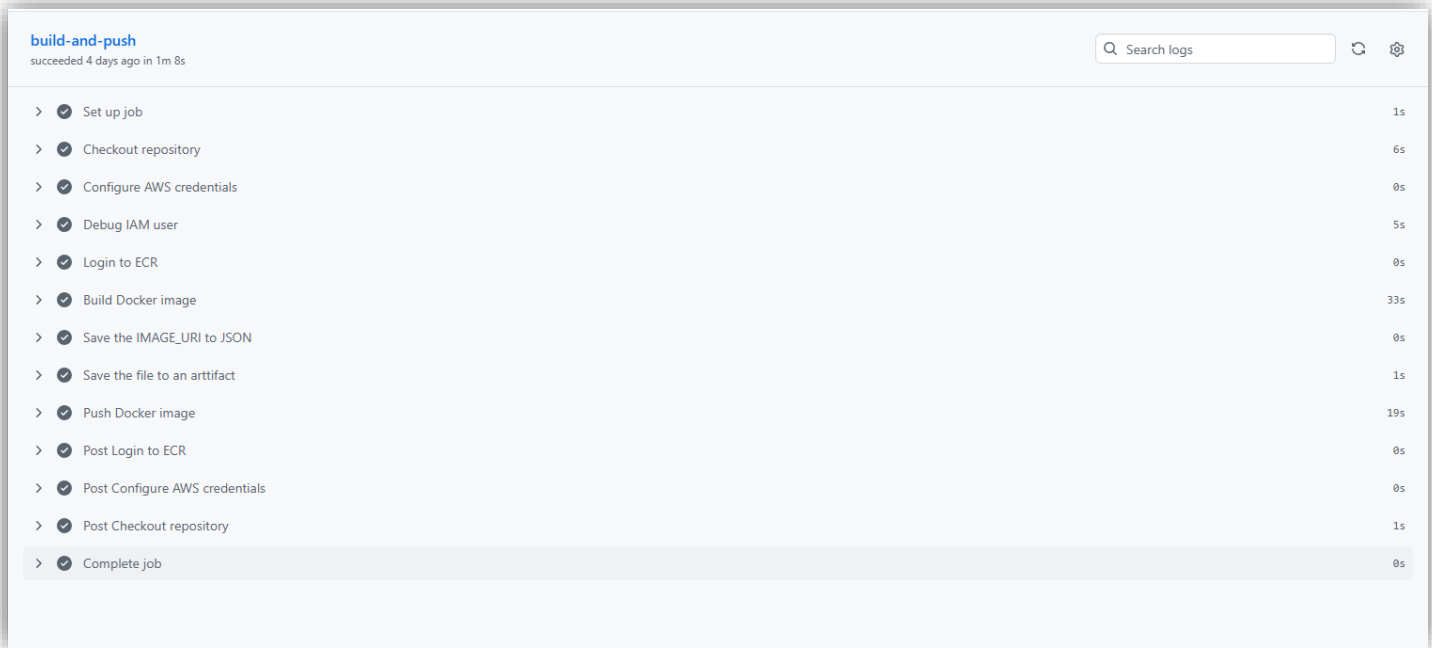


Figure 28 : L'execution de workflow-1 sous github actions

Comme etape de verification,il suffit de consulter **ECR** et verifier est ce l'image a ete cree :

Repository name	URI	Created at	Tag immutability	Encryption type
fake-news-repo	317286499743.dkr.ecr.us-east-1.amazonaws.com/fake-news-repo	November 23, 2025, 19:10:40 (UTC+01)	Mutable	AES-256

Figure 30 : Registre des images

Images (1)						Delete Copy URI Details Scan View push commands
Image tags	Type	Created at	Image size	Image digest	Last pulled at	
latest	Image	November 24, 2025, 18:27:57 (UTC+01)	315.10	sha256:5ccd21323d46190...	November 24, 2025, 18:29:10 (UTC+01)	

Figure 29 : L'image cree

• Workflow -2 :



```
name: SageMaker Deployment
on:
  workflow_run:
    workflows: ["Sage Maker Custom Container"]
    types:
      - completed
env:
  AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
  AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
  MODEL_OUTPUT: ${ secrets.MODEL_OUTPUT }
  ROLE_ARN: ${ secrets.ROLE_ARN }
  AWS_DEFAULT_REGION: ${ secrets.AWS_DEFAULT_REGION }
  MODEL_NAME: ${ secrets.MODEL_NAME }
  SAGEMAKER_ENDPOINT: ${ secrets.SAGEMAKER_ENDPOINT }
jobs:
  deploy-job:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3
      - name: Read image URI from artifact
        uses: actions/github-script@v7
        with:
```

```

script: |
  const fs = require('fs');
  const path = require('path');
  const { execSync } = require('child_process');
  const artifactName = 'image_info.json';
  const artifact = await github.rest.actions.listWorkflowRunArtifacts({
    owner: context.repo.owner,
    repo: context.repo.repo,
    run_id: context.payload.workflow_run.id
  });

  const imageArtifact = artifact.data.artifacts.find(a => a.name ===
artifactName);
  console.log('Found artifact:', imageArtifact.name);
  // Download the artifact (returns a zip)
  const download = await github.rest.actions.downloadArtifact({
    owner: context.repo.owner,
    repo: context.repo.repo,
    artifact_id: imageArtifact.id,
    archive_format: 'zip'
  });
  // Save zip to disk
  const buffer = Buffer.from(download.data, 'base64');
  const zipPath = path.join(process.cwd(), 'artifact.zip');
  fs.writeFileSync(zipPath, buffer);
  // Unzip and read JSON
  execSync(`unzip -o ${zipPath} -d ./artifact`);
  const jsonPath = path.join('./artifact', 'image.json');
  const data = fs.readFileSync(jsonPath, 'utf8');
  const json = JSON.parse(data);
  // Parse output
  console.log('IMAGE_URI:', json.image_uri);
  // Pass the variable to the next step
  fs.appendFileSync(process.env.GITHUB_ENV, `IMAGE_URI=${json.image_uri}\n`);

- name: Set up Python
  uses: actions/setup-python@v4
  with:
    python-version: 3.9

- name: Install dependencies
  run: |
    pip install "sagemaker>=2.0.0,<3.0.0" dotenv boto3

- name: Deploy model
  run: |
    echo "Using IMAGE_URI=$IMAGE_URI"
    python ML/src/deploy.py
  env :
    IMAGE_URI: ${ env.IMAGE_URI }

```

code 6 : Configuration du deployment dans SageMaker

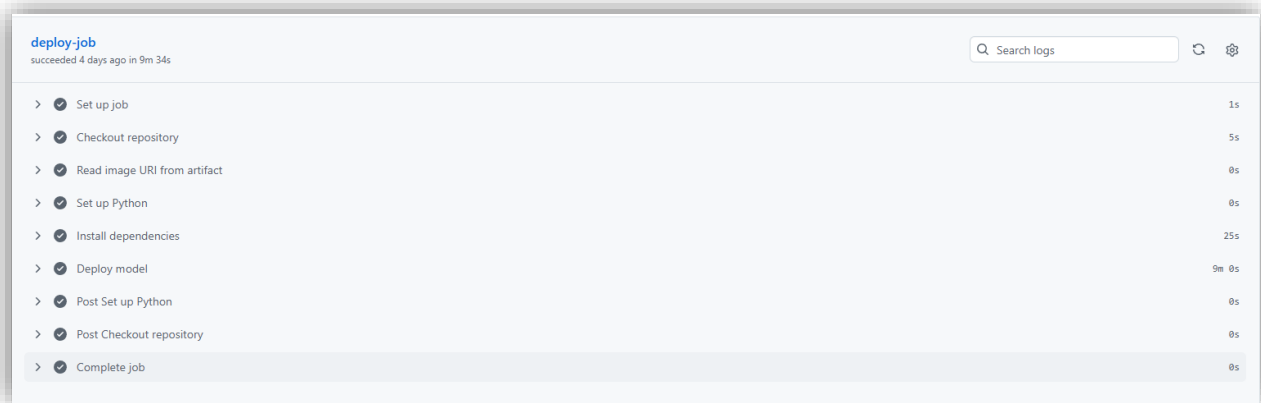


Figure 32 : l'execution de workflow-2 sous github action

Verification dans **sagemaker** :

Name	ARN	Creation time	Status	Last updated
fake-news-endpoint	arn:aws:sagemaker:us-east-1:317286499743:endpoint/fake-news-endpoint	11/24/2025, 6:29:05 PM	InService	11/24/2025, 6:37:16 PM

Figure 33 : Point de terminaison en Service

Name	ARN	Creation time
fake-news-detector	arn:aws:sagemaker:us-east-1:317286499743:model/fake-news-detector	11/24/2025, 6:29:05 PM

Figure 31 : Model Deploye

2) Deploiment d'application Web :

En parallèle du modèle, le déploiement de l'application **React** est automatisé grâce à un workflow **GitHub Actions**. À chaque modification du dépôt, le workflow est déclenché, construit l'application avec **Vite** et publie automatiquement les fichiers générés sur **Amazon S3**. Ce processus garantit que l'application reste constamment à jour et synchronisée avec les dernières modifications du code, sans intervention manuelle.

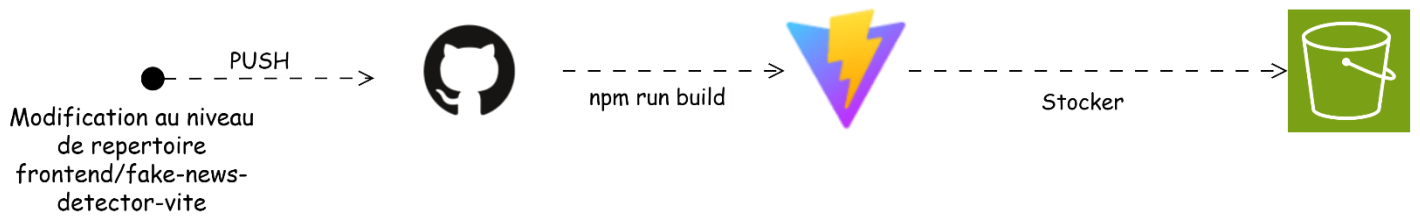


Figure 34 :Le flux automatisé pour le deployment d'une application React dans S3

• Workflow :



```
name: Frontend Deploy to S3

on:
  push:
    branches:
      - master
    paths:
      - "frontend/fake-news-detector-vite/**"

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '22.x'

      - name: Install dependencies and Build App
        run: |
          cd frontend/fake-news-detector-vite
          npm install --legacy-peer-deps
          npm run build

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v3
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ secrets.AWS_DEFAULT_REGION }

      - name: Sync build folder to S3
        run: |
          aws s3 sync frontend/fake-news-detector-vite/dist s3://${ secrets.FRONT_END_BUCKET }
      --delete
```

code 7 : Configuration du deployment d'application React

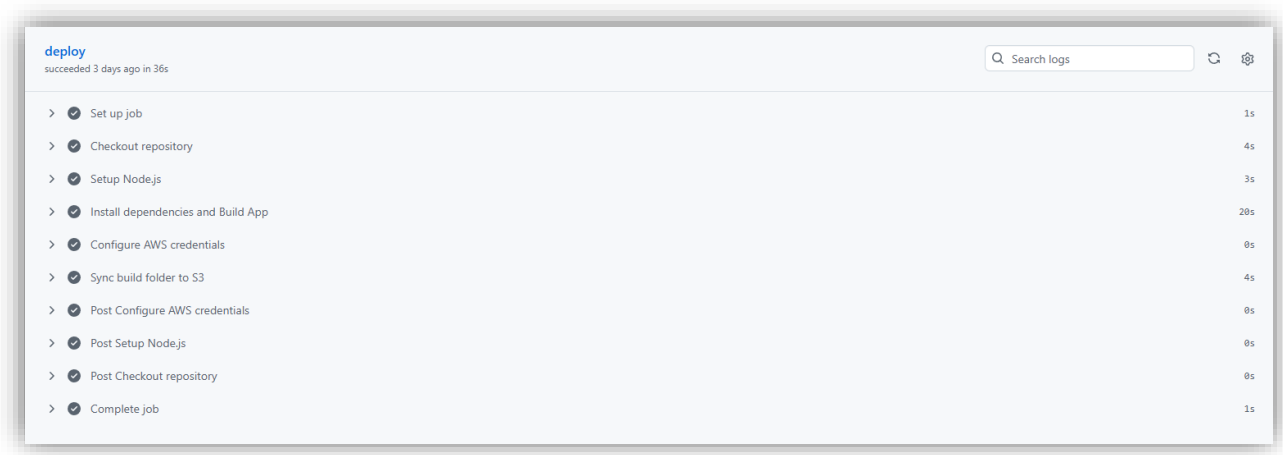


Figure 35 : l'execution de workflow-3 sous github action

Verification :

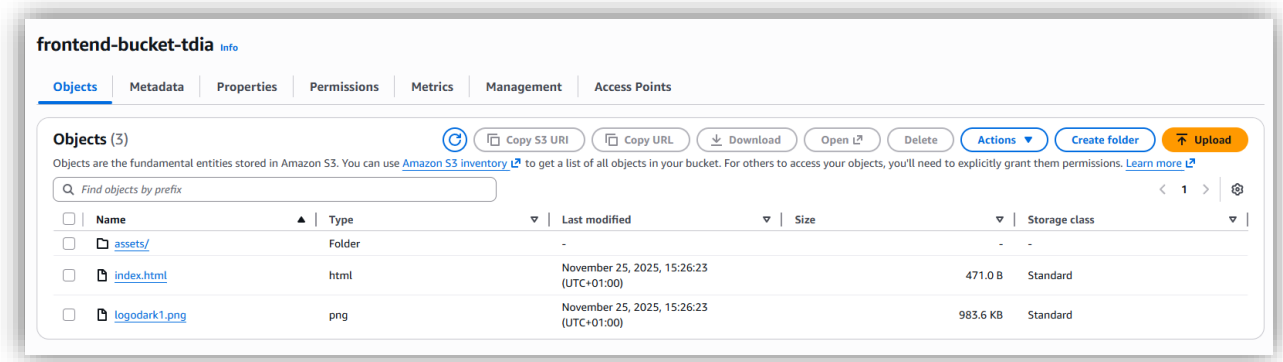


Figure 36 : Le contenu du frontend

IX. Gestion des accès et communication sécurisée entre les services :

Pour sécuriser l'accès aux ressources AWS et assurer une communication fiable entre les services, nous utilisons des rôles **IAM** associés à des politiques définissant précisément les permissions nécessaires. Grâce au mécanisme de **role assuming**, chaque service ou workflow peut temporairement assumer un rôle pour effectuer ses tâches, comme accéder à **ECR**, **S3** ou **SageMaker**. Cette approche garantit un contrôle granulaire des accès tout en maintenant une communication sécurisée et efficace entre les différents composants du pipeline **CI/CD**.

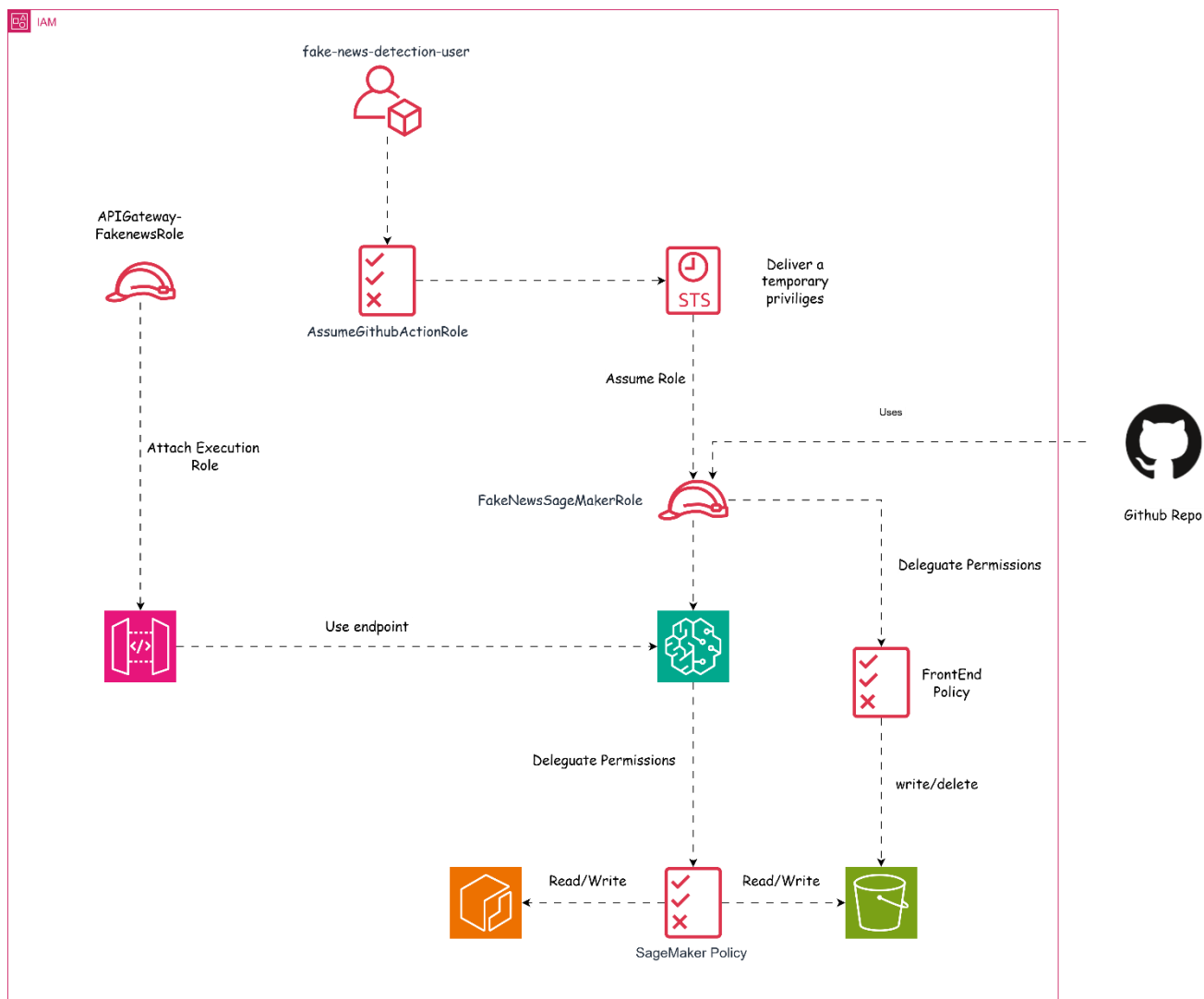


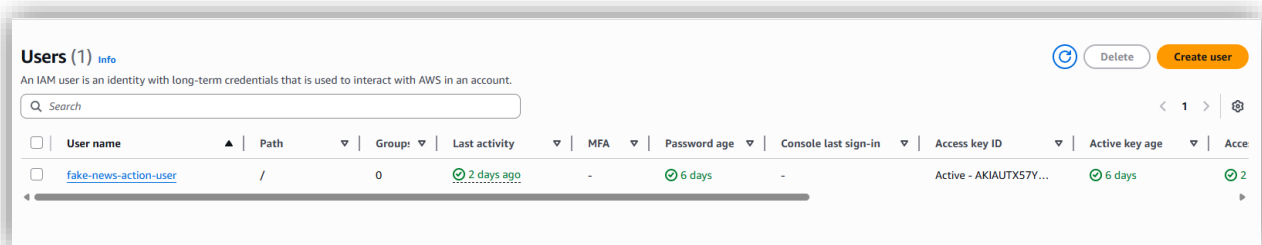
Figure 37 : Diagram de gestion d'accès AWS du projet

Le schéma illustre la gestion sécurisée des accès et des permissions dans le cadre du déploiement automatisé du modèle **SageMaker** via **GitHub Actions**. L'utilisateur IAM principal **fake-news-detection-user** délègue les droits au rôle **AssumeGitHubActionRole**, que **GitHub Actions** assume temporairement grâce au service AWS STS via une authentification OIDC. Ce rôle permet d'assumer à son tour le rôle d'exécution **FakeNewsSageMakerRole**, qui dispose des permissions nécessaires pour gérer les endpoints **SageMaker** et accéder aux buckets **S3**. Parallèlement, le rôle d'exécution **API Gateway-SageMakerRole** est utilisé pour

sécuriser et autoriser la communication entre l'**API Gateway** et **SageMaker**, garantissant que seules les requêtes authentifiées peuvent accéder aux endpoints du modèle. Grâce à cette chaîne d'assumption de rôles et aux politiques associées, chaque composant agit avec des permissions minimales et temporaires, assurant ainsi un déploiement sécurisé, automatisé et conforme aux bonnes pratiques AWS, sans exposer de clés d'accès permanentes.

1) Creation d'un utilisateur :

La création d'un nouvel utilisateur dans AWS IAM commence par accéder à la section **Users** dans le menu de gauche, puis cliquer sur **Create user**. Il faut ensuite saisir un nom d'utilisateur unique, par exemple **fake-news-detection-user**. À cette étape, il est également nécessaire de définir le **type d'accès** pour l'utilisateur en sélectionnant **Programmatic access** (pour utiliser les clés API, CLI ou SDK). ensuite il faut définir les permissions et cliquer sur **Create user**.



Specify user details

User details

User name

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . _ - (hyphen)

☐ Provide user access to the AWS Management Console - *optional*
In addition to console access, users with `SignInLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

?

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.
[Learn more](#)

Cancel

Next

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☒ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☐ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

?

Get started with groups
Create a group and select policies to attach to the group. We recommend using groups to manage user permissions by job function, AWS service access, or custom permissions.
[Learn more](#)

Create group

► Set permissions boundary - *optional*

Cancel

Previous

Next

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name	Console password type	Require password reset
test	None	No

Permissions summary

< 1 >

Name	Type	Used as
No resources		

Tags - *optional*

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create user

Figure 38 : Formulaire de creation des utilisateurs

2) Creations des roles :

Pour créer un rôle, il faut d'abord accéder à la section **Roles** puis sélectionner **Create Role** et le type de **trusted entity**. Dans notre cas, nous choisissons **AWS services**, car **SageMaker** sera chargé de communiquer avec les buckets **S3** et les images **ECR**, tandis qu'**API Gateway** sera responsable d'envoyer des requêtes vers le endpoint SageMaker. Il faut ensuite indiquer le service chargé d'exécuter les actions. La deuxième étape consiste à attribuer les permissions appropriées au rôle, définissant les actions qu'il est autorisé à réaliser. Enfin, il suffit de nommer le rôle et de cliquer sur **Create role** pour le créer.

Add permissions info

Permissions policies (1) info
The type of role that you selected requires the following policy.

Policy name	Type
AmazonSageMakerFullAccess	AWS managed

[▶ Set permissions boundary - optional](#)

[Cancel](#) [Previous](#) [Next](#)

Select trusted entity info

Trusted entity type

☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.

☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
SageMaker

Choose a use case for the specified service.

Use case

☒ **SageMaker - Execution**
Allows SageMaker notebook instances, training jobs, and models to access S3, ECR, and CloudWatch on your behalf.

☐ **SageMaker - HyperPod Clusters**
Allows SageMaker HyperPod to call AWS services on your behalf.

[Cancel](#) [Next](#)

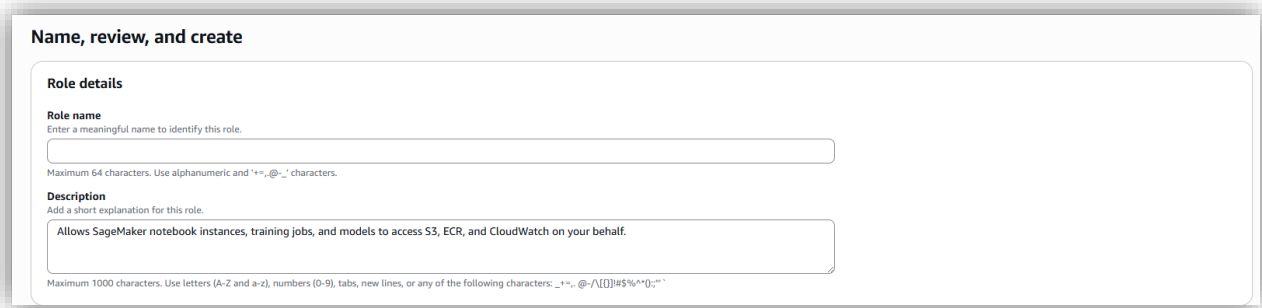


Figure 39 : Formulaire de creation d'un role

3) Creation des poltiques :

- Assumer le role d'action au SageMaker Role :

Il est nécessaire de permettre à certaines entités, comme l'utilisateur **fake-news-detector-user** d'autoriser la propagation du workflow GitHub Actions alors il faut assumer un rôle pour obtenir temporairement les permissions nécessaires.

Dans notre cas, l'utilisateur assume le rôle **FakeNewsSageMakerRole**, qui dispose des autorisations requises pour interagir avec SageMaker, accéder aux buckets S3 et aux images ECR.

Cette opération est configurée via la **trust relationship** du rôle, qui spécifie quelles entités sont autorisées à l'assumer. Lorsqu'il assume ce rôle via le service **STS**, le workflow reçoit des **credentials temporaires**, garantissant un accès sécurisé et limité dans le temps, conformément au principe du moindre privilège.

La première étape consiste à assumer le rôle **FakeNewsSageMakerRole** via le service STS. Pour ce faire, il faut accéder à l'utilisateur concerné et créer une permission dédiée, dans notre cas nommée **AssumeGitHubActionRole**. Il s'agit ensuite de définir la **policy** associée ainsi que les ressources sur lesquelles cette permission s'applique, afin de permettre à l'utilisateur ou au workflow GitHub Actions d'obtenir temporairement les droits nécessaires pour interagir avec SageMaker et les autres services AWS.



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "sts:AssumeRole",
        "sts:TagSession"
      ],
      "Resource": "arn:aws:iam::317286499743:role/FakeNewsSageMakerRole"
    }
  ]
}
```

code 8 : politics d'assumption d'un role via STS

Une étape importante de la configuration des accès consiste à définir la **trust policy** du rôle **FakeNewsSageMakerRole**, afin de permettre à différentes entités d'assumer temporairement ses permissions. Dans notre cas, cette politique autorise deux principaux acteurs. D'une part, elle permet à **GitHub Actions**, via OIDC, d'assumer le rôle uniquement pour le dépôt FakeNewsDetectionSystem, grâce à la condition spécifiant le jeton et le sous-dépôt autorisé. D'autre part, elle autorise le **service SageMaker** à assumer le rôle pour accéder aux ressources nécessaires, comme les buckets S3 et les images Docker dans ECR, lors de l'exécution d'un endpoint ou d'un job. Cette configuration garantit un accès sécurisé et temporaire aux ressources AWS, tout en respectant le principe du moindre privilège et en limitant les entités pouvant utiliser le rôle à celles explicitement autorisées.

Voici la configuration à implémenter :



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::317286499743:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub": "repo:tarik-boukaidi/FakeNewsDetectionSystem:*"
        }
      }
    },
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "sagemaker.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

code 9 : Assumption du role a github workflow action

- Définir les permissions du role :

Une étape essentielle dans la configuration des permissions consiste à définir la **policy** attachée au rôle **FakeNewsSageMakerRole**, qui précise les actions autorisées sur les services AWS.

Cette politique se compose de deux parties principales. La première accorde un ensemble complet d'actions sur **SageMaker**, permettant notamment de créer, décrire, mettre à jour et supprimer des modèles, des endpoints et des jobs d'entraînement, ainsi que d'obtenir les jetons d'authentification pour accéder aux images ECR.

La seconde partie autorise des actions sur **S3**, **ECR** et le rôle **IAM** lui-même, incluant la lecture et l'écriture d'objets dans les buckets S3, le téléchargement et l'upload d'images et de layers dans ECR, ainsi que la possibilité de passer le rôle lors des opérations nécessitant des permissions temporaires. Cette configuration assure que le rôle dispose de tous les droits nécessaires pour gérer le cycle complet du déploiement et de l'inférence du modèle tout en limitant ces droits aux ressources explicitement définies, garantissant ainsi sécurité et efficacité.



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": [
        "sagemaker:DescribeTrainingJob",
        "sagemaker:CreateModel",
        "sagemaker>DeleteEndpointConfig",
        "sagemaker>CreateEndpoint",
        "sagemaker:DescribeModel",
        "ecr:GetAuthorizationToken",
        "sagemaker:StopTrainingJob",
        "sagemaker>DeleteModel",
        "sagemaker:ListTrainingJobs",
        "sagemaker>CreateTrainingJob",
        "sagemaker:UpdateEndpoint",
        "sagemaker>CreateEndpointConfig",
        "sagemaker:DescribeEndpointConfig",
        "sagemaker>DeleteEndpoint",
        "sagemaker:DescribeEndpoint"
      ],
      "Resource": "*"
    },
    {
      "Sid": "VisualEditor1",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "iam:PassRole",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "ecr:CompleteLayerUpload",
        "s3:ListBucket",
        "ecr:UploadLayerPart",
        "ecr:InitiateLayerUpload",
        "ecr:BatchCheckLayerAvailability",
        "ecr:PutImage"
      ],
      "Resource": [
        "arn:aws:ecr:us-east-1:317286499743:repository/fake-news-repo",
        "arn:aws:s3:::fake-news-detection-tdia/*",
        "arn:aws:s3:::fake-news-detection-tdia",
        "arn:aws:iam::317286498743:role/FakeNewsSageMakerRole"
      ]
    }
  ]
}
```

code 10 : Permissions donnees au FakeNewsDetectionRole

Pour que API Gateway devra capable d'invoquer l'endpoint de SageMaker il faut lui attribuer un Role d'exécution ayant la politiques ci dessous :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": "sagemaker:InvokeEndpoint",
      "Resource": "arn:aws:sagemaker:us-east-1:317286499743:endpoint/fake-news-endpoint"
    }
  ]
}
```

code 11 : politique d'invocation de sagemaker endpoint

X. Conclusion :

Ce projet nous a permis de parcourir l'ensemble du cycle de vie d'un système complet de détection de fausses informations, depuis la préparation des données et l'entraînement du modèle jusqu'à son déploiement automatisé en production. Après avoir développé et évalué différents algorithmes, le modèle final a été emballé dans un conteneur personnalisé, puis déployé sur AWS SageMaker afin de garantir scalabilité, robustesse et performance. La mise en place d'une architecture sécurisée s'est appuyée sur API Gateway pour exposer le service d'inférence, IAM pour assurer un contrôle strict des permissions, et STS pour permettre l'assume-role depuis GitHub Actions.

L'automatisation du déploiement grâce à un pipeline CI/CD a permis d'améliorer considérablement la fiabilité du système : chaque mise à jour du code source déclenche automatiquement la génération d'un nouveau conteneur Docker, son push dans ECR, puis la mise à jour du modèle sur SageMaker. En parallèle, l'application front-end React est continuellement reconstruite et déployée sur S3, garantissant une expérience utilisateur fluide et toujours à jour.

En somme, ce travail démontre comment concevoir une solution complète mêlant machine learning, architecture cloud, sécurité et automatisation. Le système final est modulable, sécurisé et extensible, ouvrant la voie à des améliorations futures telles que l'optimisation du modèle, le monitoring en temps réel ou la mise en place d'alertes automatisées.



Lien sur **GitHub** :

<https://github.com/tarik-boukaidi/FakeNewsDetectionSystem.git>